



PDF Download
3785469.pdf
04 April 2026
Total Citations: 0
Total Downloads: 168

Latest updates: <https://dl.acm.org/doi/10.1145/3785469>

RESEARCH-ARTICLE

CoSrch: Syntactic and Semantic Structure-Aware Code Representation for Effective Code Search

JIA XU, Guangzhou University, Guangzhou, Guangdong, China

YUQIANG HE, Guangxi University, Nanning, Guangxi, China

HENGYU LIU, Aalborg University, Aalborg, Nordjylland, Denmark

PIN LV, Guangzhou University, Guangzhou, Guangdong, China

TIANCHENG ZHANG, Northeastern University, Shenyang, Liaoning, China

ZHIHONG TIAN, Guangzhou University, Guangzhou, Guangdong, China

Open Access Support provided by:

Guangzhou University

Northeastern University

Aalborg University

Guangxi University

Published: 24 December 2025

Accepted: 20 August 2025

Revised: 29 June 2025

Received: 07 December 2024

[Citation in BibTeX format](#)

CoSrch: Syntactic and Semantic Structure-Aware Code Representation for Effective Code Search

JIA XU, Cyberspace Institute of Advanced Technology in Guangzhou University; State Key Laboratory of Cyberspace Security Defense (Institute of Information Engineering, Chinese Academy of Sciences); Guangdong Key Laboratory of Industrial Control System Security; Huangpu Research School of Guangzhou University, China
YUQIANG HE, School of Computer, Electronics and Information, Guangxi University, China

HENGYU LIU, Department of Computer Science, Aalborg University, Denmark

PIN LV*, Cyberspace Institute of Advanced Technology in Guangzhou University, Guangzhou, China

TIANCHENG ZHANG, School of Computer Science and Engineering, Northeastern University, China

ZHIHONG TIAN, Cyberspace Institute of Advanced Technology, Guangzhou University, Guangdong Key Laboratory of Industrial Control System Security, and Huangpu Research School of Guangzhou University, China

Code search, which retrieves relevant code snippets from a large codebase based on natural language queries, significantly enhances software development efficiency and quality, with effective code representation being crucial for its success. While recent studies leverage both syntactic and semantic structures of code snippets to improve representation, they inadequately address two key challenges: (1) inadequate capture of long-range dependencies in these structures, and (2) ineffective fusion of syntactic and semantic information. To overcome these limitations, we propose CoSrch, a deep model for syntactic and semantic-aware code representation. CoSrch first encodes code snippets as graphs to enable structured alignment and fusion, then employs a GNN-based graph encoder to capture long-range dependencies (addressing Challenge 1), along with an overlap-aware modality decomposition and fusion framework that eliminates redundancy when fusing the syntactic and semantic information (resolving Challenge 2). Extensive experiments on prominent benchmarks demonstrate CoSrch's superiority, achieving at least a 7.60% improvement in SuccessRate@1 and a 5.41% gain in MRR on the CSN-Java dataset compared to the best baseline, validating its effectiveness in advancing code search performance.

CCS Concepts: • **Software and its engineering** → **Software development techniques**.

Additional Key Words and Phrases: Code search, code representation, deep networks, syntactic and semantic structures of codes

Authors' Contact Information: Jia Xu, Cyberspace Institute of Advanced Technology in Guangzhou University; State Key Laboratory of Cyberspace Security Defense (Institute of Information Engineering, Chinese Academy of Sciences); Guangdong Key Laboratory of Industrial Control System Security; Huangpu Research School of Guangzhou University, Guangzhou, China, xujia@gzhu.edu.cn; Yuqiang He, School of Computer, Electronics and Information, Guangxi University, Nanning, Guangxi, China, heyq1314@gmail.com; Hengyu Liu, Department of Computer Science, Aalborg University, Aalborg, Denmark, heli@cs.aau.dk; Pin Lv, Cyberspace Institute of Advanced Technology in Guangzhou University, Guangzhou, Guangdong, China, lvpin@gzhu.edu.cn; Tiancheng Zhang, School of Computer Science and Engineering, Northeastern University, Shenyang, China, tczhang@mail.neu.edu.cn; Zhihong Tian, Cyberspace Institute of Advanced Technology, Guangzhou University, Guangdong Key Laboratory of Industrial Control System Security, and Huangpu Research School of Guangzhou University, Guangzhou, China, tianzhihong@gzhu.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s).

ACM 1557-7392/2025/12-ART

<https://doi.org/10.1145/3785469>

1 Introduction

The rapid growth of open-source software communities, especially through platforms such as GitHub¹ and Stack Overflow², has significantly enhanced the accessibility and availability of large-scale shared code repositories. Empirical studies [46, 57] demonstrate that more than 90% of software development efforts focus on reusing existing code. Given this overwhelming reliance on code reuse, effective code search mechanisms which enable developers to efficiently locate relevant code snippets within those extensive code repositories are crucial for improving software productivity and quality while simultaneously reducing development costs [27, 50].

Early code search methods [9, 33, 41, 47, 50, 51] primarily rely on traditional information retrieval (IR) techniques, such as sparse vector retrievers (e.g., TF-IDF and BM25 [54]). However, these approaches treat both queries and codes as plain texts and simply match source code snippets based on the presence of query keywords, thereby neglecting the crucial semantics embedded within both queries and codes (e.g., function names and variable roles in a code snippet). The advent of deep learning (DL) has played a pivotal role in capturing the semantic information of both code and natural language queries through advanced neural network architectures [8, 21, 66]. Specifically, most modern deep code search methods employ sequential models, such as convolutional neural networks (CNNs) [38], long-short-term memory networks (LSTMs) [59], self-attention mechanisms [61], and co-attention mechanisms [6], to learn semantic representations of queries and codes separately and then measure their semantic relevance using cosine similarity. While the semantic features of queries and code snippets have been successfully extracted, sequential models often neglect the syntactic structure of code snippets, which is crucial for reflecting their functional characteristics [7]. To address this issue, recent studies [15, 19, 24] have proposed representing the syntactic structure of code snippets as Abstract Syntax Trees (ASTs) [52]. Then, for each code snippet, its AST-based syntactic structure is encoded using graph neural networks (GNNs) [55], while its semantic features are encoded using a sequence encoder, after which the two aspects of encoding results are merged to derive a unified representation of the code snippet.

However, existing deep code search models are still limited in two major aspects, reducing their search performance.

- **Challenge 1: Inadequate capture of long-range dependencies in code structure.** Long-range dependencies in code syntactic (or semantic) structure are essential for capturing data and control flow across the code, thereby enhancing models' ability to identify patterns and context, and improving code understanding [44, 71]. However, recent attempts to address this challenge by increasing the depth of GNNs that model graph-based code structures [49, 63] have inadvertently led to two critical issues: over-squashing [3] and over-smoothing [40]. These two issues degrade the models' ability to learn differentiated node representations, ultimately affecting the quality of the generated code representations.
- **Ineffective fusion of code syntactic and semantic information.** For a given code snippet, the syntactic structure is typically represented as an AST graph, while semantic features are organized as a sequence. The inherent heterogeneity between these two representation modalities [21] poses significant challenges for effective integration, requiring sophisticated alignment mechanisms. Despite recent advances [13, 22, 65], existing fusion strategies fail to adequately address semantic redundancy between the graphical features and sequential features, leading to overlapping information [48] and reduced discriminative power in the final code representations.

To address the aforementioned challenges, this paper presents a novel syntactic and semantic-aware deep network named *CoSrch*, which effectively captures and fuses the syntactic and semantic structures of every code snippet so as to provide a more accurate representation of it. In specific, given a query in natural language towards a code base, CoSrch first constructs the semantic graphs for both the query and each code snippet in the

¹<https://github.com>

²<https://stackoverflow.com>

base using dependency parsing methods [32]. Concurrently, the syntactic graph of each code snippet is extracted in the form of a abstract syntax tree (AST). Note that, in CoSrch, the syntactic and semantic structures of a code snippet are both modeled as graphs. This unified representation facilitates the alignment and fusion of the two structures in subsequent steps. Then, to address the two challenges facing by recent works, CoSrch designs the *overlap-aware modality decomposition and fusion framework (OA-MDFF)* which contains two phases, i.e., *overlap-aware modal decomposition (OA-MD)* and *multimodal fusion (MF)*. In specific, to address Challenge 1, two GNN-based graph encoders are designed and deployed in the OA-MD phase, each of which is responsible to encode the syntactic (or semantic) graph of a code snippet. Note that a encoder can capture both short- and long-range dependencies in the syntactic (or semantic) structure of the code snippet by considering interactions among different k -hop neighbor graphs of the code syntactic (or semantic) graph. This approach avoids learning the long-range dependencies via the local aggregation paradigm of GNNs, which can produce less discriminative node representations. To address Challenge 2, three loss functions are firstly proposed in the OA-MD phase which target at extracting the overlapping and non-overlapping information from the two modalities (i.e., syntactic and semantic graphs), separately. Subsequently, in the MF phase, two decoders are designed: one decoder focuses on deriving the overlapping information ($F_{\cap}$), while the other computes the non-overlapping information (F_{-}) from the decomposed outputs of the OA-MD phase, thereby reducing redundancy between modalities. Finally, the final representation for the code snippet are derived by combining $F_{\cap}$ and F_{-} . To evaluate the effectiveness of the proposed CoSrch model in code search tasks, we compare it with 11 state-of-the-art models using C, Java, Python and JavaScript datasets. Three standard evaluation metrics are employed: **SuccessRate@k**, **MRR**, and **Precision@k**. Experimental results demonstrate that CoSrch achieves significant performance improvements compared to all baselines. Meanwhile, ablation studies verify the effectiveness of each individual module in CoSrch. The source code of CoSrch is publicly available at: <https://github.com/sincerity-being/CoSrch>.

The main contributions of this work are:

- We propose CoSrch, a novel syntactic and semantic-aware deep network for effective code search. Different from previous works, both the syntactic and semantic structures of a code snippet are consistently modeled by graphs, which facilitates the follow-up alignment and fusion of these two modalities.
- CoSrch introduces an overlap-aware modality decomposition and fusion framework (OA-MDFF) to enhance code representation learning. On one hand, OA-MDFF captures both short- and long-range dependencies in syntactic (or semantic) graphs by learning interactions across different k -hop neighbor graphs of the code syntactic (or semantic) graph, effectively addressing Challenge 1. On the other hand, it eliminates redundancy between syntactic and semantic graphs before fusing them, resolving Challenge 2. By tackling these key limitations of prior works, CoSrch improves the final representations of each code snippet, leading to higher-quality code search results.
- Extensive experiments are carried out on multiple prominent code search datasets respectively corresponding to several widely-used programming languages, e.g., Java and Python. The experimental results deliver that the proposed CoSrch model surpasses all state-of-the-art methods in terms of handling code search task.

The remainder of this paper is organized as follows: Section 2 presents motivating examples of the proposed CoSrch model. Section 3 introduces preliminaries, including syntactic and semantic graph construction and the formalized problem studied in this work. Section 4 elaborates the implementation of the CoSrch model. Section 5 presents the experimental setup and reports the results. Section 6 reviews related works. Finally, Section 7 concludes the paper and points out future research directions.

2 Motivating Examples

This section presents running examples to demonstrate the effectiveness of our CoSrch model in addressing the syntax- and semantics-aware code search challenges outlined in Section 1.

Recent advances in GNNs [49, 67] have demonstrated their effectiveness in modeling short-range node dependencies within code snippet graphs, yet propagating long-range dependencies remains a persistent challenge for traditional GNN-based learning [58] which learns the node dependencies by the local aggregation paradigm, i.e., iteratively aggregating information from immediate neighbors via message propagating mechanisms. In such case, increasing GNN depth could theoretically capture long-range node dependencies. However, the deeper architecture of GNNs suffers from two critical issues: over-squashing [3] and over-smoothing [40]. The over-squashing issue refers to the distortion and compression of information from distant nodes during message propagation, severely limiting the model’s ability to capture long-range dependencies. On the other hand, the over-smoothing issue occurs when repeated message-propagating iterations cause node representations to become indistinguishable, losing their discriminative power. Since long-range dependencies between nodes in code semantic or syntactic graphs of a code snippet are crucial for accurately modeling program logic (e.g., nested block dependencies), these two issues fundamentally prevent GNNs from effectively learning long-range dependencies in the code structure graphs. Consequently, this limitation degrades the quality of learned code representations and ultimately compromises the performance of code search tasks.

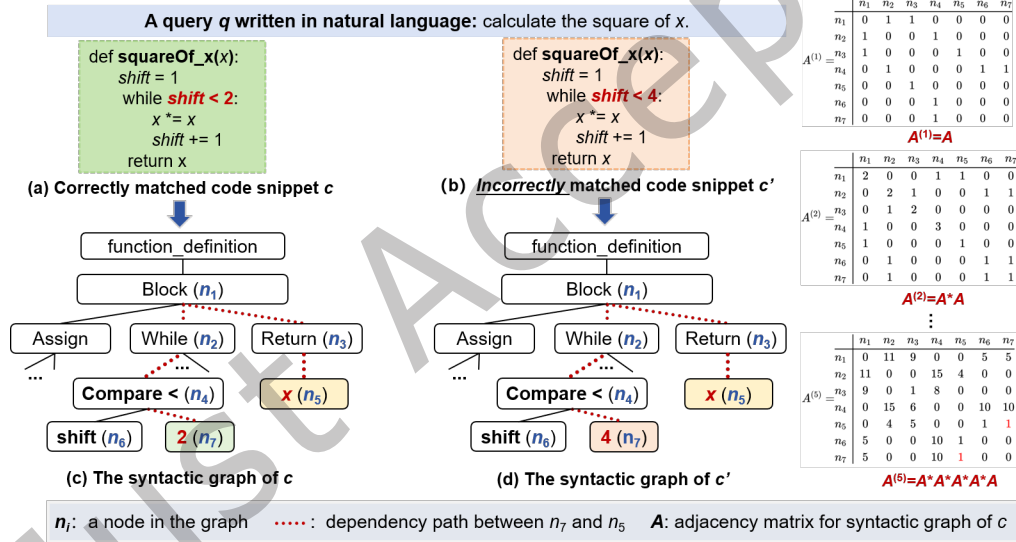


Fig. 1. An example of matching code snippets for a query.

Taking the query “calculate the square of x ” in natural language as an example which is listed at the top of Figure 1. Two candidate code snippets in a code base, i.e., c and c' , are separately illustrated in subfigures 1(a) and 1(b). As shown by these two subfigures, the two code snippets c and c' , are very similar, except for the setting of the loop condition in the while statement: in c , the loop condition is set to $shift < 2$, while in c' , it is set to $shift < 4$. Due to this single difference, it is snippet c , not c' , that matches the query q , because c' returns not the square but the eighth power of the variable x . Returning code snippets that accurately match the semantics of query q is non-trivial, as it requires accurately capturing the dependency between the return value of function `squareOf_x` and the loop bound (i.e. 2 for c and 4 for c'). Subfigures 1(c) and 1(d) separately demonstrate the

extracted AST-based syntactic graph of c and c' . These graphs reveal a long-range dependency path (i.e., $length=5$) connecting the function's return value (node n_7) to the loop bound (node n_5): $n_7 \rightarrow n_4 \rightarrow n_2 \rightarrow n_1 \rightarrow n_3 \rightarrow n_5$. This demonstrates that modeling long-range dependencies in code structure graphs is essential for correctly identifying code functionality. However, if the local aggregation paradigm in GNNs is applied over the graph, due to the aforementioned over-squashing and over-smoothing issues, such long-range dependencies may be lost or diluted during message passing. This thus compromises the model's ability to distinguish between semantically critical differences (e.g., loop bounds), leading to incorrect matches for queries like q in Figure 1.

To achieve the goal of effectively learning short- and long-range node dependencies in the syntactic graph of a code snippet (i.e., *Challenge 1*), instead of using the local aggregation paradigm of GNNs, CoSrch proposes to learn such two types of dependencies by considering the interactions between different k -hop neighbor graphs of the code structure graph. The rightmost part of Figure 1 depicts the adjacency matrices of 1-hop, 2-hop, and 5-hop neighbor graphs of the code snippet c (i.e., $A^{(1)}$, $A^{(2)}$, and $A^{(5)}$), where a k -hop neighbor graph is the result of the k^{th} power of the adjacency matrix A . It is noteworthy that the value of each element in a matrix $A^{(k)}$ denotes the number of k -hop paths between the corresponding two nodes in the graph. For example, the element $A^{(5)}[n_7][n_5]$ indicates that there is a 5-hop path from node n_7 to node n_5 . Then, following the idea of the graph attention network (GAT), CoSrch builds a k -hop interaction graph by treating every k -hop neighbor graph as a node and using edges to model their interactions, which enables to learning of the importance of dependency relationships in different k -hop neighbor graphs for identifying the functionality of the code snippet. By directly modeling short- and long-range dependencies through the inputted k -hop neighbor graphs instead of relying on GNNs' local aggregation paradigm, CoSrch effectively learns multi-scale dependencies across the graph structure. The incorporated attention mechanism further enhances code snippet representation by adaptively weighting dependencies of varying lengths, ultimately improving performance in subsequent code search tasks.

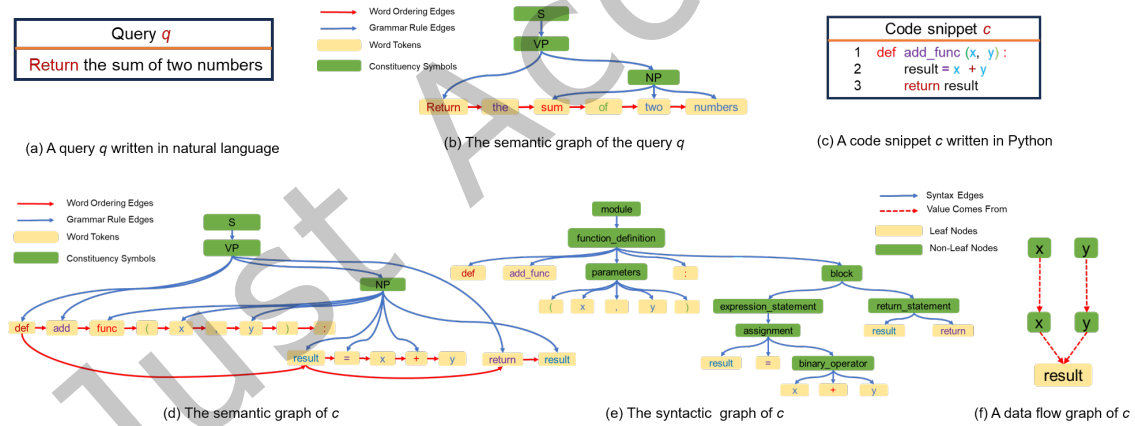


Fig. 2. An example of different constructed structural information for a query or code snippet.

Figure 2 illustrates different constructed structural information for a query q or code snippet c . As seen in Figure 2(d) and Figure 2(e), many nodes (e.g., "x", "y", "result", and "return") appear in both the semantic and syntactic graphs of the code snippet c . This overlap occurs because the AST structure [53], which encodes syntactic relationships, inherently captures partial semantic information [14, 20]. However, this redundancy makes a straightforward fusion strategy of the information in two graphs very ineffective, as they fail to sufficiently emphasize the distinct features of each graph, leading to suboptimal fusion performance [70]. To

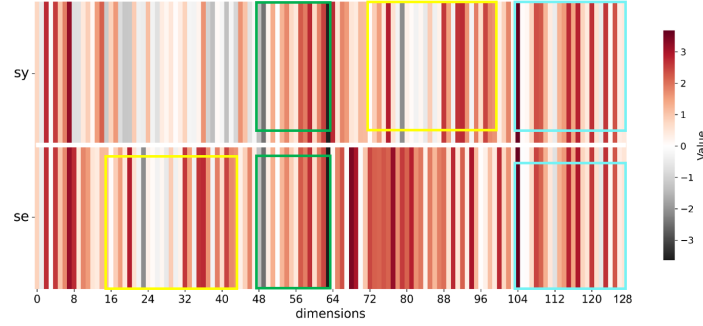


Fig. 3. Visualization for vector representations of the syntactic and semantic graphs of code snippet *c* computed by graph encoders (**sy**: syntactic; **se**: semantic; similar parts in two vectors are framed by the rectangles of the same color).

examine the effect of information redundancy between code syntactic and semantic modalities on multimodal fusion effectiveness, in Figure 3, we visualize the vector representations of the syntactic graph (row 1) and semantic graph (row 2) of code snippet *c* (shown in Figure 2(d)-(e)) computed by the proposed graph encoders (Section 4.1). Figure 3 reveals that (1) the two vectors share multiple similar segments (highlighted by same-color rectangles), yet (2) these segments are spatially misaligned in the vector space. Consequently, additive fusion amplifies destructive interference, as redundant but misaligned features weaken modality-specific signals, while concatenation-based fusion compounds redundancy, increasing noise and reducing the discriminative power of the fused representation—ultimately undermining model performance by obscuring critical distinctions between syntactic and semantic structures.

Seeing this, inspired by CodeDisen [68] and CDDFuse [70], a novel overlap-aware modality decomposition and fusion framework (OA-MDFF) is carefully designed in the proposed CoSrch model, which well handles the overlapping and non-overlapping information in the semantic and syntactic graphs of a code snippet. By doing so, CoSrch improves the information exchange between different modalities of the code snippet (i.e., semantic and syntactic), compared with recent related works.

3 Preliminaries

Section 3.1 explains the generation of syntactic (or semantic) graph for a query (or code snippet). Subsequently, the code search task is formally defined from a unified perspective of the graph in Section 3.2.

3.1 Syntactic and Semantic Graph Construction

Inspired by [44], the graph has a more powerful ability to model rich semantic structure information from a code snippet than the sequential structure. Consequently, in the CoSrch model, the semantic structure of a query in a natural language representation or a code snippet is also constructed as a graph (called the semantic graph), while the syntactic structure of a code snippet has been generally modeled by the AST.

3.1.1 Syntactic Graph Construction. In our solution, the syntactic graph of a code snippet is represented by the AST [53], which is built following the idea of [1]. The AST is a tree representation of the syntactic structure of the code snippet. Specifically, the syntactic graph based on AST of a code snippet is formally denoted as $\mathcal{G}^{\Delta_c} (\mathcal{V}^{\Delta_c}, \mathcal{E}^{\Delta_c})$, where $\mathcal{V}^{\Delta_c}$ is a set of nodes in the AST, and $\mathcal{E}^{\Delta_c}$ symbolize the edges that delineate relationships between nodes. As illustrated in Figure 2(e), every node in the constructed syntactic graph $\mathcal{G}^{\Delta_c}$ for the code snippet shown in Figure 2(c) signifies a distinct structure in the code. Note that a leaf node in $\mathcal{G}^{\Delta_c}$ can represent a

variable (i.e., “x”), a function (i.e., “func”), an operator (i.e., “+”), or a literal (i.e., “1”), while an intermediate node denotes various compilation units, such as “block”, “assignment”, and “binary_operator”.

3.1.2 Semantic Graph Construction. The constituency parse tree [32], which models composition and ordering information of sentences, is commonly employed for extracting semantic structure from natural language texts [49]. Considering the proposed queries of users over a code base are generally represented in the form of natural languages, extracting the semantic structure from the queries using the idea of constituency parse tree is very natural. With respect to the semantic structure of a code snippet, existing works propose to model such information using a data flow graph (DFG) [23]. Figure 2(f) illustrates the constructed DFG of the code snippet in Figure 2(c). However, we argue that a DFG-based solution has some limitations. On one hand, DFG-based solution can only capture the variable value flow and fails to model the semantic control structure of a code snippet which is vital information delivered by the code. On the other hand, when the semantic structure of a user query is extracted by a constituency parse tree, representing the semantic structure of code snippet by a DFG leads to the semantic understanding gap between these two structures. In view of two limitations, in this paper, we propose to consistently adopt the constituency parse tree to build the semantic graph for each code snippet, due to the resemblance between codes and natural languages [10, 26]. Specifically, the semantic graph for a query q (or a code snippet c) is denoted by $\mathcal{G}^q = (\mathcal{V}^q, \mathcal{E}^q)$ (or $\mathcal{G}^c = (\mathcal{V}^c, \mathcal{E}^c)$), where $\mathcal{V}$ is a set of nodes with each node representing a token in q (or c) and $\mathcal{E}$ is a set of edges with every edge denoting the relation between two connected tokens in q (or c). For example, considering the query q = “Return the sum of two numbers” displayed in Figure 2(a), the semantic graph for the query is shown in Figure 2(b). As indicated by the figure, **S** serves as the designated root symbol, an intermediate node represents the grammatical properties of verbs (i.e., **VP**) or the composition of nouns (i.e., **NP**), and a leaf node corresponds to a token in the text sequence of the query q . Meanwhile, the red edge in the graph represents the word ordering relation between two tokens in the text sequence of the query, while the blue edge linking an intermediate node and a leaf node denotes the grammar rule relation between them. The constructed semantic graph for the code snippet in Figure 2(c) is provided in Figure 2(d). Figure 2(b) and Figure 2(d) illustrate that the semantic graph for a query or (a code snippet) built in the proposed CoSrch model summarizes all implicit semantic relationships [69] (i.e., word ordering and grammar rules) in it.

3.2 Problem Formulation

The syntactic and semantic structure-aware code search problem studied in this paper is formally defined below.

Definition 3.1. Syntactic and Semantic Structure-Aware Code Search Problem. Let q be a code retrieval query written in natural language and C be a code base having $|C|$ code snippets c . Assume that the semantic graph of query q (or each code snippet $c \in C$), i.e., $\mathcal{G}^q = (\mathcal{V}^q, \mathcal{E}^q)$ (or $\mathcal{G}^c = (\mathcal{V}^c, \mathcal{E}^c)$), is constructed and the syntactic graph of every code snippet $c \in C$, i.e., $\mathcal{G}^{\Delta c}(\mathcal{V}^{\Delta c}, \mathcal{E}^{\Delta c})$, is also built. Let each of the constructed graph be represented by its adjacency matrix $\mathbf{A}(\mathcal{G}) \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ with $\mathbf{A}(\mathcal{G})_{i,j} = 1$ if $\text{edge}(i, j) \in \mathcal{E}$, and 0 otherwise. Moreover, $\mathbf{X}(\mathcal{G}) \in \mathbb{R}^{|\mathcal{V}| \times d}$ represents the feature matrix of $|\mathcal{V}|$ nodes in graph $\mathcal{G}$, where $\mathbf{x}(\mathcal{G})_{ij} \in \mathbf{X}(\mathcal{G})$ denotes the j^{th} feature of the i^{th} node in the graph and d is the dimension of a node’s feature vector. In addition, some notations are summarized in Table 1. Then, the goal of the syntactic and semantic structure-aware code search problem over C is to retrieve the most relevant code snippet $\hat{c} \in C$ with respect to a query q based on the information of $\mathcal{G}^q$, $\mathcal{G}^c$, and $\mathcal{G}^{\Delta c}$, i.e.,

$$\hat{c} = \arg \max_{c \in C} \mathbf{rel}(q, c) = \arg \max_{c \in C} \mathbf{rel}(\mathcal{G}^q, f(\mathcal{G}^c, \mathcal{G}^{\Delta c})) \quad (1)$$

Here, $\mathbf{rel}(q, c)$ is the function that computes the relevant score between query q and code snippet c . As indicated by Equation 1, during the computation of the relevant score, the query q is represented by its semantic graph $\mathcal{G}^q$, and the code snippet c is represented by the fusion of its syntactic graph and semantic graph, i.e., $f(\mathcal{G}^c, \mathcal{G}^{\Delta c})$.

Table 1. Frequently-used notations and their descriptions

Notations	Descriptions
$\mathcal{G}^q$	The query semantic graph of the query q .
$\mathcal{G}^c$ (or $\mathcal{G}^{\Delta c}$)	The semantic (or syntactic) graph of a code snippet c .
$\bar{\mathcal{G}}_{0...K}^i$	The K -hop interaction graph for node n_i built by fully connecting k -hop neighbor graphs ($0 \leq k \leq K$) of a code structure graph $\mathcal{G}$.
$X(\mathcal{G})$ (or $A(\mathcal{G})$)	The node feature matrix (or adjacency matrix) of graph $\mathcal{G}$.
$A^k(\mathcal{G})$	The k^{th} order adjacency matrix of graph $\mathcal{G}$.
α, β, γ	The weights of decomposition loss $\mathcal{L}_{de}$, semantic reconstruction loss $\mathcal{L}$, and syntactic reconstruction loss $\mathcal{L}_{\Delta}$, separately.
$E(\mathcal{G})$	The embedding of graph $\mathcal{G}$.
F_c	The embedding of a code snippet that encodes its semantic structure and syntactic structure information.
F_q	The embedding of a query that encodes its semantic structure information.

4 Our Approach

In this section, we outline the CoSrch workflow and provide a detailed description of each module's structure. The framework integrates query semantic graphs and code semantic (or syntactic) graphs, as illustrated in Figure 4. It comprises four sequential modules: syntactic and semantic graph construction, overlap-aware modality decomposition and fusion framework (OA-MDFF), query semantic encoder, and code search. As syntactic and semantic graph construction is discussed in Section 3.1, we focus here on the remaining three modules. Given that the graph encoder resolves Challenge 1 and involves technical complexity, we present its details at first in a dedicated section.

4.1 Graph Encoder

Graph encoder is presented to efficiently capture short- and long-range node dependencies and facilitates the learning of discriminative node representations in the syntactic/semantic code graphs (i.e., addressing *Challenge 1*). As illustrated in Figure 5, the graph encoder encodes a syntactic (or semantic) graph and is composed by three steps, namely k -hop neighbor graph embedding, k -hop neighbor graph interaction, and attention layer.

4.1.1 k -Hop Neighbor Graph Embedding. Extracting long-range neighbor information is critical for nodes to achieve representative representations [11]. While conventional node interaction GNNs that rely on stacking K layers to reach K -hop neighbors at the cost of over-squashing [3] and over-smoothing issues [40], our graph encoder adopts a nonparametric hop-aware pre-embedding strategy. Given a code structure graph $\mathcal{G}$ (semantic or syntactic), we decompose it into distinct k -hop neighbor graphs $\{\mathcal{G}_k\}_{k=0}^K$, where K represents the maximum hop and $\mathcal{G}_k$ captures the k -hop neighborhood relationships as shown in Figure 5. For example, $\mathcal{G}_2$ contains edges

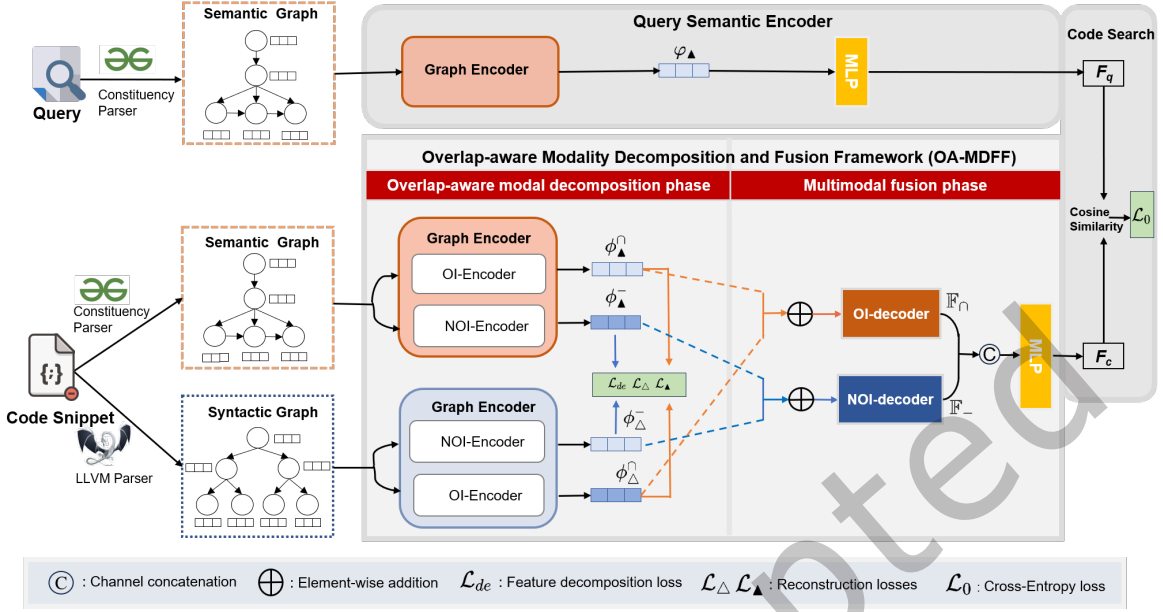


Fig. 4. The framework of CoSrch.

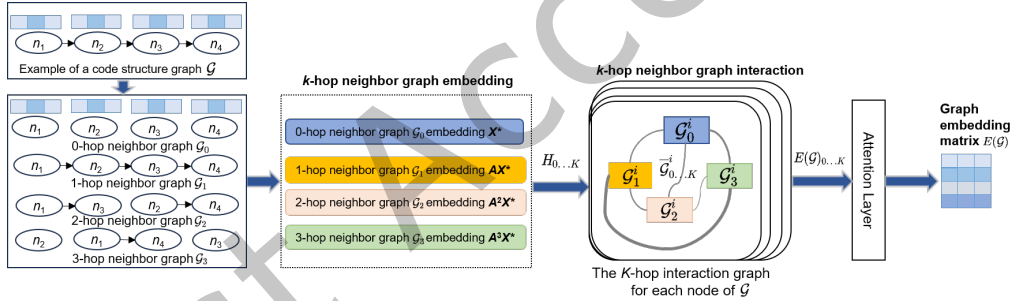


Fig. 5. The process steps of the graph encoder.

between node pairs like \$(n_1, n_3)\$ that are connected through two-hop dependency paths. Let \$\mathbf{A}(\mathcal{G}) \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}\$ denote the adjacency matrix of \$\mathcal{G}\$, where \$|\mathcal{V}|\$ represents cardinality of nodes. The \$k\$-hop neighbor graph embedding are computed by the matrix power operations:

$$\mathbf{H}_k = \mathbf{A}^k(\mathcal{G})\mathbf{X}^* \quad \text{where} \quad \mathbf{X}^* \in \mathbb{R}^{|\mathcal{V}| \times d} \quad (2)$$

Here, \$\mathbf{X}^*\$ represents randomly initialized node embeddings (0-hop embedding) and \$d\$ denotes the embedding dimension. The complete \$K\$-hop embedding \$\mathbf{H}_{0...K} \in \mathbb{R}^{|\mathcal{V}| \times (K+1) \times d}\$ concatenates all hop-specific embeddings:

$$\mathbf{H}_{0...K} = [\mathbf{H}_0 \| \mathbf{H}_1 \| \dots \| \mathbf{H}_K] \quad (3)$$

In summary, the core idea of embedding a \$k\$-hop neighbor graph is matrix multiplication, which does not rely on any learnable parameter, making it scalable for handling large-scale graphs.

4.1.2 k -hop Neighbor Graph Interaction. Instead of applying the local aggregation paradigm of GNNS to learn dependencies of nodes in a code structure graph, this module proposes to learn such dependencies by modeling nonlinear interactions across different k -hop neighbor graphs, so as to ensure the discriminative representation power of the CoSrch model towards code snippets while balancing local-global contexts. To model the interactions across different k -hop neighbor graphs, for every node n_i in the code structure graph, a K -hop interaction graph $\bar{\mathcal{G}}_{0...K}^i$ is constructed by treating each of its k -hop neighbor graph $\mathcal{G}_k^i$ ($0 \leq k \leq K$) as the node and using full-connected edges to represent their interactions. Then, a stack of L GNN layers with residual connections are built over the K -hop interaction graph $\bar{\mathcal{G}}_{0...K}^i$. Equation 4 lists the learning details of k -hop neighbor graph interactions in the underling GNN structure.

$$\begin{aligned} \mathbf{E}_0^i &= \mathbf{H}_{0...K}^i, \\ \mathbf{E}_l^i &= \text{GNN}(\mathbf{E}_{l-1}^i) + \mathbf{E}_{l-1}^i, \quad l = 1 \dots L, \end{aligned} \quad (4)$$

where $\mathbf{H}_{0...K}^i \in \mathbb{R}^{(K+1) \times d}$ is the embedding of the K -hop interaction graph $\bar{\mathcal{G}}_{0...K}^i$ for node n_i , $\mathbf{E}_l^i \in \mathbb{R}^{(K+1) \times d}$ denotes the embedding for node n_i after the l^{th} message passing in the GNN. In specific, we implement $\text{GNN}(\cdot)$ in Equation 4 using a standard GAT [62] to capture complex interactions across different k -hop neighbor graphs of node n_i . Implementation details of $\text{GNN}(\cdot)$ is listed in Equations 5 to 8:

$$e_{k,k'} = \text{LeakyReLU}(\omega^T [\mathbf{W}_s \cdot h_k \oplus \mathbf{W}_s \cdot h_{k'}]), \quad (5)$$

$$e_{k,k'}^* = \frac{\exp(e_{k,k'})}{\sum_{n_{k'}' \in \mathcal{N}(n_k')} \exp(e_{k,k''})}, \quad (6)$$

$$h_k^* = \sigma \left(\sum_{n_{k'}' \in \mathcal{N}(n_k')} e_{k,k'}^* \cdot \mathbf{W}_e \cdot h_{k'} \right), \quad (7)$$

$$\text{GNN}(\mathbf{E}^i) = [h_0^* \| h_1^* \| \dots \| h_K^*], \quad (8)$$

where we omit the superscript l of layers for simplicity. Here, $\mathbf{W}_s \in \mathbb{R}^{d \times d}$, $\omega \in \mathbb{R}^{2d \times d}$, and $\mathbf{W}_e \in \mathbb{R}^d$ are learnable parameters; h_k and $h_{k'} \in \mathbb{R}^d$ represent embedding vectors of node n_k' and its neighboring node $n_{k'}'$ in $\bar{\mathcal{G}}^i$; $\oplus$ denotes concatenation operation; and $\mathcal{N}(n_k')$ is the neighbor set of n_k' in $\bar{\mathcal{G}}_{0...K}^i$. Notice that the embedding for every node n_i in the code structure graph $\mathcal{G}$ (i.e., $\mathbf{E}^i \in \mathbb{R}^{(K+1) \times d}$) is obtained by concatenating the updated embeddings of all nodes in $\bar{\mathcal{G}}_{0...K}^i$ which is shown in Equation 8.

Then, the final output of the k -hop neighbor graph interaction step in a graph encoder is represented as $\mathbf{E}(\mathcal{G})_{0...K}$, which is the concatenation of the embeddings of all nodes in the code structure graph $\mathcal{G}$, i.e.,

$$\mathbf{E}(\mathcal{G})_{0...K} = \bigoplus_{i=1}^{|\mathcal{V}|} \mathbf{E}^i \in \mathbb{R}^{|\mathcal{V}| \times (K+1) \times d}, \quad (9)$$

where $|\mathcal{V}|$ denotes the number of nodes in $\mathcal{G}$ and $\bigoplus$ denotes the concatenation operation.

4.1.3 Attention Layer. Given $E(\mathcal{G})_{0...K}$ as the input, during the step of attention layer, the input is processed by a self-attention mechanism [61], so as to derive the relative importance of different k -hop neighbor graphs (i.e., different lengths of node dependencies in the code structure graph $\mathcal{G}$). After that, the final embedding of the input code structure graph $\mathcal{G}$ to the graph encoder, denoted as $E(\mathcal{G})$, is computed according to Equation 10 below.

$$E(\mathcal{G}) = \text{einsum}(Y, E(\mathcal{G})_{1...K}) + E(\mathcal{G})_0, \quad (10)$$

$$\text{where } Y = \text{softmax}(\sigma(W_0 E(\mathcal{G})_0 + \text{sq}(W_{1...K} E(\mathcal{G})_{1...K}))). \quad (11)$$

Here, $E(\mathcal{G})_0 \in \mathbb{R}^{|\mathcal{V}| \times d}$ represents the 1st row in the matrix $E(\mathcal{G})_{0...K}$, $E(\mathcal{G})_{1...K} \in \mathbb{R}^{|\mathcal{V}| \times K \times d}$ contains the 2nd row to the $(K+1)^{\text{th}}$ row in $E(\mathcal{G})_{0...K}$, and $W_0 \in \mathbb{R}^{d \times K}$ together with $W_{1...K} \in \mathbb{R}^{d \times 1}$ are learnable matrices. Meanwhile, $\text{sq}(\cdot)$ is employed to eliminate dimensions of a matrix that equal 1 in the shape, $\sigma(\cdot)$ represents the activation function, and $\text{einsum}(\cdot)$ denotes the Einstein summation convention [11] which transforms two input matrices with the shapes of $[|\mathcal{V}|, K]$ and $[|\mathcal{V}|, K, d]$ respectively into an output matrix with the shape $[|\mathcal{V}|, d]$.

4.2 Overlap-aware Modality Decomposition and Fusion Framework

To address *Challenge 2* outlined in Section 1 regarding the ineffective fusion of syntactic and semantic information in code snippets, the proposed CoSrch model introduces the an overlap-aware modality decomposition and fusion framework (OA-MDFF). OA-MDFF systematically integrates the syntactic and semantic structure graphs of every code snippet while preserving modality-specific characteristics and minimizing inter-modal redundancy [48]. The architecture of OA-MDFF mainly contains two phases, that is, overlap-aware modal decomposition (OA-MD) which disentangles modality overlap and modality-specific representations, and multimodal fusion (MF) that fuses the disentangled modality-specific and shared representations among modalities.

4.2.1 Overlap-aware Modal Decomposition. The two modalities, i.e., the semantic structure graph and the syntactic structure graph of a code snippet, are encoded separately using the graph encoder introduced in Section 4.1 before being fused. To capture both overlapping and distinct information between the two modalities (as illustrated in Figure 4), two instances of the graph encoder are employed for each modality: the overlapping information encoder (*OI-encoder*) and the non-overlapping information encoder (*NOI-encoder*). Given the semantic (or syntactic) structure graph, the outputs of the OI-encoder are denoted as $\phi^\cap \in \mathbb{R}^{|\mathcal{V}| \times d}$ (or $\phi_\Delta^\cap \in \mathbb{R}^{|\mathcal{V}| \times d}$), while the outputs of the NOI-encoder are represented as $\phi^- \in \mathbb{R}^{|\mathcal{V}| \times d}$ (or $\phi_\Delta^- \in \mathbb{R}^{|\mathcal{V}| \times d}$).

To compute $\phi^\cap$, ϕ^- , $\phi_\Delta^\cap$, and ϕ_Δ^- , three loss functions are defined: a feature decomposition loss ($\mathcal{L}_{de}$) as given in Equation 12, a semantic reconstruction loss ($\mathcal{L}$) detailed in Equation 13, and a syntactic reconstruction loss ($\mathcal{L}_\Delta$) outlined in Equation 14.

$$\mathcal{L}_{de} = \frac{(\mathcal{L}_{CC}^-)^2}{\mathcal{L}_{CC}^\cap} = \frac{(CC(\phi_\Delta^-, \phi^-))^2}{CC(\phi_\Delta^\cap, \phi^\cap) + \varepsilon}, \quad (12)$$

where $CC(\cdot, \cdot)$ is the correlation coefficient operator, and $\varepsilon = 1.01$ ensures that the denominator remains positive [70].

$$\mathcal{L} = \mathcal{L}_{euc}^I(I, I') + \mathcal{L}_{cos}^I(I, I'); \quad (13)$$

$$\mathcal{L}_\Delta = \mathcal{L}_{euc}^V(V, V') + \mathcal{L}_{cos}^V(V, V'); \quad (14)$$

$$I = \phi^\cap + \phi^-, \quad I' = \phi_\Delta^\cap + \phi_\Delta^-; \quad (15)$$

$$V = \phi_\Delta^\cap + \phi_\Delta^-, \quad V' = \phi^\cap + \phi^-. \quad (16)$$

Here, $\mathcal{L}_{euc}^I = \|I - I'\|^2$, $\mathcal{L}_{cos}^I = 1 - \text{cosine}(I, I')$, $\mathcal{L}_{euc}^V = \|V - V'\|^2$, and $\mathcal{L}_{cos}^V = 1 - \text{cosine}(V, V')$.

Note that the overlapping and non-overlapping information between two modalities (i.e., syntactic and semantic structures) is separated iteratively during the training of CoSrch rather than being predefined. Initially, the graph encoders may produce representations that fail to distinguish modality-shared (overlapping) and modality-specific (non-overlapping) information. For example, early outputs of the OI-encoder or NOI-encoder might still contain mixed signals. As training advances, the three loss functions ($\mathcal{L}_{de}$, $\mathcal{L}$, and $\mathcal{L}_{\Delta}$) sharpen this separation. $\mathcal{L}_{de}$ pushes $\phi^{\cap}$ and $\phi_{\Delta}^{\cap}$ to capture shared details (e.g., leaf nodes), while ϕ^{-} and ϕ_{Δ}^{-} focus on modality-specific features (e.g., semantic edges or syntactic grammar rules). Simultaneously, the reconstruction losses ($\mathcal{L}$ and $\mathcal{L}_{\Delta}$) ensure the encoded representations preserve the original code snippet's information through encoding and decoding and the non-overlapping and overlapping information within a single modal's encoded representation is gradually expanded [68]. This iterative process enables the graph encoder to progressively disentangle redundant (overlapping) and non-redundant (non-overlapping) information across semantic and syntactic graphs.

4.2.2 Multimodal Fusion. As shown in Figure 4, our architecture processes decomposed features through dual fusion pathways. The overlap components ($\phi^{\cap}$, $\phi_{\Delta}^{\cap}$) of the structural modalities of the code snippet undergo fusion through an overlapping information decoder (*OI-decoder*), while distinct components (ϕ^{-} , ϕ_{Δ}^{-}) are processed through a non-overlapping information decoder (*NOI-decoder*). Both decoders employ residual network architectures [25], with their outputs formalized as:

$$\mathbf{F}_{\cap} = \text{OI-Decoder}(\phi^{\cap}, \phi_{\Delta}^{\cap}). \quad (17)$$

$$\mathbf{F}_{-} = \text{NOI-Decoder}(\phi^{-}, \phi_{\Delta}^{-}). \quad (18)$$

Finally, the derived $\mathbf{F}_{\cap}$ and $\mathbf{F}_{-}$ are concatenated and then fed into a subsequent MLP layer to generate the embedding of the code snippet, represented as $\mathbf{F}_c \in \mathbb{R}^d$ (see Equation 19 for details).

$$\mathbf{F}_c = \text{MLP}(\mathbf{F}_{\cap} \oplus \mathbf{F}_{-}). \quad (19)$$

4.3 Query Semantic Encoder

Section 4.2 utilizes the graph encoder proposed in Section 4.1 to compute the representation of a code snippet (i.e., $\mathbf{F}_c \in \mathbb{R}^d$) that considers both its semantic structure and syntactic structure information. Since the user-submitted query for a code searching task is written in natural language, it does not contain information on the syntactic structure. As shown in Figure 4, in query semantic encoder, only the semantic structure graph is constructed based on the query text following the idea in Section 3.1. Then, a graph encoder presented in Section 4.1 is applied to encode the query semantic structure graph and compute the query semantic vector φ . The derived query semantic vector φ is then fed into an MLP layer to compute the final representation of the user query, denoted as $\mathbf{F}_q \in \mathbb{R}^d$.

4.4 Syntactic and Semantic Structure-aware Code Searching

The essence of the syntactic and semantic structure-aware code search task, defined in Section 3.2, lies in the calculation of the relevant score between a query q and a code snippet c , i.e., $\mathbf{rel}(q, c)$. In the implementation of the proposed CoSrch model, the relevant score between q and c is computed as the cosine similarity between their representations $\mathbf{F}_q$ and $\mathbf{F}_c$, i.e.,

$$\mathbf{rel}(q, c) = \mathbf{rel}(\mathcal{G}^q, f(\mathcal{G}^{\Delta c}, \mathcal{G}^c)) = \text{cosine}(\mathbf{F}_q, \mathbf{F}_c). \quad (20)$$

4.5 Model Training

As indicated in Figure 4, there are three types of loss functions, i.e., $\mathcal{L}_{de}$, $\mathcal{L}$, and $\mathcal{L}_{\Delta}$, defined in the OA-MDFF of the proposed CoSrch model. Specifically, the feature decomposition loss $\mathcal{L}_{de}$, defined in Equation 12, aims to disentangle shared and modality-specific features across multiple modalities of code snippets, thereby guiding

the feature decomposition process. The two reconstruction losses, namely the semantic reconstruction loss $\mathcal{L}$ listed in Equation 13 and the syntactic reconstruction loss $\mathcal{L}_\Delta$ shown in Equation 14, are proposed to primarily ensure that the information in a code snippet is preserved during the encoding and decoding process. Meanwhile, following the typical loss setting of training a deep code search model [49], a Cross-Entropy loss $\mathcal{L}_0$ is defined in CoSrch in Equation 21. Specifically, given n query-code snippet pairs $\langle q_i, c_i \rangle, 1 \leq i \leq n$, $\mathcal{L}_0$ aims to minimize the inner product between the query q_i and its corresponding code c_i , while simultaneously maximizing the inner product between the query q_i and other distractor codes c_j , i.e., c_j where $j \neq i$.

$$\mathcal{L}_0 = -\frac{1}{n} \sum_i \log\left(\frac{c_i^T q_i}{\sum_j c_j^T q_i}\right) \quad (21)$$

Finally, the overall training loss $\mathcal{L}$ of the proposed CoSrch model is the weighted aggregation of the aforementioned four loss components, i.e.,

$$\mathcal{L} = \mathcal{L}_0 + \alpha \cdot \mathcal{L}_{de} + \beta \cdot \mathcal{L} + \gamma \cdot \mathcal{L}_\Delta, \quad (22)$$

where α , β , and γ are learnable weight parameters. To ensure high-quality trainable parameters α , β , and γ , we adopt a dual optimization framework combining constrained initialization and structural regularization. First, we implement a bounded uniform initialization scheme during early training phases, which establishes balanced priors by restricting parameter values to reasonable ranges, thus avoiding extreme initial conditions that could hinder optimization. Second, we introduce a softmax-based parameterization mechanism (Equation 23) [34] that enforces two critical mathematical properties: strict positivity ($\alpha, \beta, \gamma > 0$) and unit summation ($\alpha + \beta + \gamma = 1$). This architectural constraint serves two key purposes: it prevents pathological solutions where parameters degenerate to zero or infinity, which would effectively disable corresponding loss components, while maintaining stable gradient flow throughout training. The dual optimization framework ensures all loss terms contribute meaningfully to the optimization process, with the softmax transformation providing inherent numerical stability and the initialization scheme establishing robust starting conditions for subsequent learning.

$$\alpha = \frac{\exp(w_1)}{\sum_{i=1}^3 \exp(w_i)}, \quad \beta = \frac{\exp(w_2)}{\sum_{i=1}^3 \exp(w_i)}, \quad \gamma = \frac{\exp(w_3)}{\sum_{i=1}^3 \exp(w_i)}. \quad (23)$$

Here, $w_i \in \mathbb{R}$ ($i = 1, 2$, or 3) is a learnable parameter.

5 Experiments and Results

This section experimentally evaluates CoSrch's performance by addressing specific research questions:

- **RQ1:** Can CoSrch outperform existing baselines in terms of code search effectiveness?
- **RQ2:** How do different graph representations in CoSrch impact its performance?
- **RQ3:** How do the graph encoder and k -hop neighbor graph interaction-based learning affect CoSrch's performance?
- **RQ4:** How is the generalization ability of CoSrch for different code search scenarios?
- **RQ5:** How much does *OA-MDFF* contribute to CoSrch's effectiveness, and how does its redundancy removal mechanism influence model performance?
- **RQ6:** How do three types of loss functions (i.e., $\mathcal{L}_{de}$, $\mathcal{L}$, and $\mathcal{L}_\Delta$) in CoSrch impact its performance?
- **RQ7:** What are CoSrch's training and inference efficiency?
- **RQ8:** How effective is CoSrch in handling real-world queries compared to baselines?

5.1 Experimental Setup

Experimental setup is elaborated in this section.

Table 2. Cardinality of the training, validating, and testing subdatasets of each dataset.

Dataset	Training	Validating	Testing
Python	294,417	16,026	15,906
Java	261,483	8,208	14,738
JavaScript	1,051,824	60,314	75,340
C	51,251	6,120	5,545

5.1.1 Datasets. To evaluate our model’s effectiveness, we use four physical-world datasets corresponding to four mainstream programming languages which are briefly named as *Python*, *Java*, *JavaScript* and *C*. Specifically, Python and Java are extracted from the CodeSearchNet benchmark datasets [29], the JavaScript dataset comes from [37], and the C dataset is self-generated by crawling GitHub. In each of these dataset, each sample pairs a code function together with its natural language description. For the Python, Java, and JavaScript datasets, a preprocessing involving the systematic filtering is executed to ensure high-quality of samples: (1) removing code with parsing errors that hinder AST generation, (2) excluding non-English documentation comments, and (3) discarding entries where code/documentation length or graph node counts fall outside the 3–300 range. For the C dataset, we collect high-star C-language projects from GitHub and curate it by extracting C methods along with their corresponding descriptions. Specifically, we include methods with comments exceeding 300 characters and code exceeding 2,000 tokens, to ensure the dataset meets the requirement of scalability analysis. Note that we generate semantic graphs for code snippets or queries based on the idea of the constituency parse trees [32] on four datasets. As to the generation of syntactic graph for every code snippet, publicity tools are used, i.e., Tree-sitter³ for Python (or JavaScript), Features-javac⁴ for Java, and LLVM-IR for C [39]. The scale of training, validating, and testing subdatasets of every dataset are listed in in Table 2, while important statistics of all datasets are summarized in Table 3. Apparently, the JavaScript dataset owns the largest samples (e.g., 1,051,824 for training), according to Table 2. Meanwhile, table 3 indicates that compared to other datasets, the C dataset produces the largest graph in scale (e.g., averagely 4,037 nodes and 4,676 edges for a graph) within four datasets.

Table 3. Statistics of graph sizes of each dataset.

Dataset		Code Syntactic Graph			Code Semantic Graph			Query Semantic Graph		
		min	max	avg	min	max	avg	min	max	avg
Java	Node #	55	285	141	23	299	73	3	300	17
	Edge #	54	313	156	22	460	90	2	422	42
Python	Node #	10	241	106	19	300	68	3	300	13
	Edge #	9	480	181	18	430	82	2	410	26
JavaScript	Node #	72	377	213	34	346	166	6	315	94
	Edge #	71	514	230	33	488	182	5	471	121
C	Node #	2317	9148	4037	1102	8070	3608	328	1180	584
	Edge #	2316	10082	4676	1101	9214	4290	327	1348	689

³<https://github.com/tree-sitter/tree-sitter>⁴<https://github.com/acr31/features-javac>

5.1.2 Evaluation Metrics. Critical to evaluating code search models is measuring the relevance of retrieved code snippets to their corresponding queries. To achieve this, three domain experts with more than 10 years of programming experience are firstly asked to construct test cases for every query in the test set, which are then executed on our self-developed online judge (OJ) platform [31]. In this case, the relevance of a retrieved code snippet is objectively assessed by its ability to pass test cases. If a retrieved code snippet can not be executed or produces incorrect output, we defer to expert manual relevance assessment, following established practices [21, 47]. During the expert assessment process, any discrepancies between different experts are resolved via discussion until consensus is achieved focusing exclusively on the model’s top-5 results. This hybrid evaluation approach, combining automated verification with expert validation, ensures an optimal balance between evaluation efficiency and precision. Leveraging these relevance annotations, we quantify model performance using three standard metrics: Mean Reciprocal Rank (MRR), SuccessRate@ k , and Precision@ k .

MRR. MRR is calculated as the average of the reciprocal ranks of the queries Q . The reciprocal rank of a query is defined as the inverse of the rank of its first hit result.

$$\text{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{FRank}_q} \quad (24)$$

SuccessRate@ k . SuccessRate@ k represents the percentage of queries for which the ground truth code method appears in the top- k ranked list. For a set of queries Q , the calculation of SuccessRate@ k is as follows:

$$\text{SuccessRate@}k = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \delta(\text{FRank}_q \leq k) \quad (25)$$

Here, δ is an indicator function that returns 1 if the ground-truth code of the i_{th} query is included in the top- k code snippets in the recommendation list; otherwise, it returns 0.

Precision@ k . It evaluates the quality of the top- k retrieved results by measuring the proportion of relevant items within the top- k results, which is computed as the ratio of relevant items in the top- k results to k :

$$\text{Precision@}k = \frac{\text{Number of relevant items in Top-}k \text{ results}}{k} \quad (26)$$

5.1.3 Parameter Settings. To configure our model, we embed a maximum of 150,000 of the most frequent tokens from both the code and query texts in the training set into 128-dimensional vectors. We apply a dropout rate of 0.3 after the word embedding layer during training. For multiple-graph encoding in parsing Java and Python, we set the hop for both to 4. Furthermore, we set the parameters α , β and γ are treated as learnable parameters in Equation 22. The ReLU activation function is utilized for the linear layer. Though tuning the number of GAT layers in the graph encoder often leads to better results, we fix the interaction layer to 2 for simplicity [11] on both Java and Python datasets, while the final output vector dimension is set to 128. We employ Adam [35] for optimization and apply LayerNorm [5] to each layer while tuning the other hyperparameters. We initialize the learning rate to 0.001 and reduce it by a factor of 0.0005 when the improvement rate falls below 0.01, ultimately determining the optimal learning rate to be 0.003. The gradient is clipped at a length of 10.0. We set the maximum number of epochs at 6 and terminate the experiment if the **MRR** fails to improve for three consecutive epochs. The size of the training batch is fixed at 1,000, consistent with previous research [29]. A batch containing 1,000 samples means that one correct code c_i corresponds to the current query q_i , while the remaining 999 code snippets serve as distractors to differentiate from the current q_i . We implement CoSrch using PyTorch 1.13.1 on a Linux system with a GPU (A800-80G) and a CPU with 256GB of RAM.

5.1.4 Baselines. We compare the proposed CoSrch model with four categories of baselines: 1) **information retrieval (IR) based models** (i.e., CodeMatcher [47]); 2) **sequential models** (i.e., CARLCS-CNN [57], DeepCS [21],

UNIF [8], and SemEr [14]); 3) **multi-modal models** (i.e., CAT [24], TabCS [66], and GraphSearchNet [49]); and 4) **pre-trained based multi-modal models** (i.e., CodeBERT [17], GraphCodeBERT [23], and UniXcoder [13]). Detailed descriptions of each of these baseline approaches are listed below.

- **CodeMatcher** [47] is the information retrieval-based approach for code search. Initially, it gathers metadata for query words to filter out irrelevant noise. Subsequently, it iteratively conducts fuzzy search based on important words towards a code repository. Finally, it generates a set of candidate code snippets based on the token-level similarity comparison between query and code snippets in the code repository.
- **CARLCS-CNN** [57] employs CNN and LSTM networks, coupled with a co-attention mechanism, to acquire interdependent representations for code and query following the individual embedding process.
- **DeepCS** [21] utilizes RNN and MLP networks on method names, API sequences, and code tokens to generate a representative vector of code. Additionally, it employs RNN to derive the representative vector of the query.
- **UNIF** [8] employs a learned attention-based weighting scheme to merge per-token embeddings, resulting in an embedded code sequential vector. Additionally, the description sequential embedding is generated by averaging the bag of query embeddings.
- **SemEr** [14] enriches the semantics of all code snippets in both training and testing data by estimating the syntactic similarity of each code snippet from the training data and retrieving the most similar one for each. Additionally, it integrates a multi-perspective co-attention mechanism, utilizing CNNs to capture local correlations between code and query.
- **CAT** [24] employs a sequence encoder to integrate two modalities, i.e., syntactic and semantic features, extracted from code tokens and their corresponding AST representations.
- **TabCS** [66] fuses two modalities, i.e., code textual features, including method names, API sequences, and tokens, along with AST, and query features (tokens), using a two-stage attention network to improve representation for both code and query.
- **GraphSearchNet** [49] suggests building graphs for both the source code and queries utilizing bi-directional Graph Gated Neural Networks (BiGGNN) [42] to capture the local structural information of both. Additionally, it enhances BiGGNN by integrating a multi-head attention module to capture global information within the code sequences, thereby enhancing the model's learning capacity.
- **CodeBERT** [17] is the first pre-trained model designed for natural language-programming language pairs, using both masked language modeling and replacement token detection in training, though it fails to integrate structural code information into its learning framework. Specifically, we employ the officially released CodeBERT implementation with default hyperparameters to conduct full-parameter fine-tuning across our four datasets.
- **GraphCodeBERT** [23] builds upon CodeBERT by introducing a graph-guided masked attention mechanism to integrate code structural information. Unlike conventional approaches that depend on syntactic-level structures like ASTs, this model uses data flow analysis in pre-training to capture semantic-level code structures. To ensure fair comparison of GraphCodeBERT and CoSrch, we develop four GraphCodeBERT variants. **GraphCodeBERT (Co-T)** freezes a subset of parameters to approximately match CoSrch's parameter count, followed by full retraining from scratch on our datasets for code search validation; **GraphCodeBERT (Full-T)** maintains all parameters trainable, undergoing complete retraining from scratch. **GraphCodeBERT (Co-F)** preserves pre-trained weights while fine-tuning with parameter constraints aligned to CoSrch's architecture. **GraphCodeBERT (Full-F)** retains pre-trained weights but performs unconstrained full-parameter fine-tuning.
- **UniXcoder** UniXcoder [13] is an encoder-decoder model that uses masked attention matrices with prefix adapters to control model behavior, enhancing code representation by integrating ASTs and code

comments. Two variants of UniXcoder are compared. **UniXcoder (Co-F)** retains pre-trained weights but applies parameter constraints aligned with CoSrch’s architecture when fine-tuning. **UniXcoder (Full-F)** preserves pre-trained weights but undergoes unconstrained full-parameter fine-tuning.

5.2 Experimental Results

5.2.1 RQ1 (Can CoSrch outperform existing baselines in terms of code search effectiveness?) We evaluate CoSrch against more than 10 baselines for code search using two benchmark datasets. Table 4 shows the results ($R@1/R@5$ = SuccessRate@ k at $k=1/5$), revealing key findings:

Finding 1: The proposed CoSrch surpasses all benchmarks on both datasets. Specifically, CoSrch achieves a minimum improvement of 7.60% for $R@1$ and 5.41% for MRR in Java (except for pre-trained models), and achieves at least a 3.42% increase in $R@1$ and a 2.77% increase in MRR for Python (except for pre-trained models). These results underscore the efficacy of our approach in addressing the code search challenge.

Finding 2: The performance of CodeMatcher exceeds that of DeepCS, CARLCS-CNN, and UNIF, aligning with the conclusions drawn in its original work [47]. However, CodeMatcher still lags behind CoSrch, underscoring the importance of extracting latent structures from both code snippets and queries to improve the model’s capacity for learning semantic mappings.

Finding 3: We find that most deep learning models, which depend exclusively on sequence-based modalities, perform less effectively than CodeMatcher. However, SemEtr surpasses both these models and CodeMatcher in performance. We posit that SemEtr’s enhanced efficacy stems from its capacity to discern semantic mappings between code and query texts through the estimation of syntactic similarities among code snippets within the training data, thereby retrieving the most analogous snippet for each query. This underscores the limitations of relying solely on sequential code structures to capture the nuanced semantics linking code and summaries, thus highlighting the necessity for supplementary information to accurately represent code semantics.

Finding 4: Although models like CAT and TabCS incorporate both semantic and syntactic information, their performance still falls short of some pure sequence models. This does not imply that syntactic information is detrimental, as demonstrated by GraphSearchNet, which effectively integrates both types of information using graph neural networks. Its advantage lies in better handling the tree-like dependencies in code. However, SemSynCode outperforms GraphSearchNet, indicating that simply combining sequential and structural information may be counterproductive. The innovation of SemSynCode lies in simultaneously constructing semantic and syntactic structure graphs, precisely aligning them, and employing the OA-MDFF module to eliminate inter-modal redundancy, thereby highlighting the strengths of each feature type and significantly improving code representation quality.

Finding 5: While large pre-trained models like CodeBERT, GraphCodeBERT, and UniXcoder have advanced code representation learning via unsupervised approaches, our carefully designed CoSrch achieves comparable or superior performance by more effective code structure learning and eliminating cross-modal redundancy. The experiments, conducted under identical conditions with standardized test settings, reveal critical insights about contemporary code search approaches. CodeBERT’s underperformance relative to GraphSearchNet (confirming prior findings) and UniXcoder’s superiority over GraphCodeBERT highlight that multi-modal contrastive learning outperforms structural analysis alone. The GraphCodeBERT variants comparison (Full-F vs. Co-T/Full-T) further emphasizes the compounded impact of model scale and training data volume on performance. While UniXcoder (Full-F) achieves marginally better results on Python/Java datasets due to its substantial parameter size (135+ MB) and extensive pretraining, CoSrch remains highly competitive despite its more compact architecture. Crucially, CoSrch surpasses GraphCodeBERT (Full-F) in effectiveness despite the latter’s larger capacity, and significantly outperforms similarly-sized pretrained models (GraphCodeBERT-Co-F, UniXcoder-Co-F). This superior performance derives from CoSrch’s dual innovations: (1) advanced graph-based modeling that preserves

both local and global node dependencies in code structures, and (2) sophisticated cross-modal fusion that eliminates redundant information while preserving modality-specific features (Figure 3). The Transformer-based baselines' limitations become apparent in their need to linearize graph structures into sequences (disrupting inherent relationships) and their inability to filter inter-modal redundancy - architectural constraints that ultimately degrade their code search effectiveness. These findings collectively validate that specialized architectural designs like CoSrch can achieve state-of-the-art performance without relying solely on model scale or pretraining volume, offering more efficient alternatives to monolithic pretrained models for code search tasks. The results particularly emphasize the critical importance of proper structural representation and intelligent feature fusion in developing high-performance code search systems.

Finding 6: Table 4 also shows that Python exhibits a superior overall performance compared to Java, primarily due to its emphasis on code conciseness and readability, as well as its simpler structure in contrast to Java. Codes written in Python more closely resemble natural language queries. Consequently, the semantic mapping between natural language queries and codes is more straightforward for Python than Java.

Table 4. Comparison with baselines (best values are in bold and next-best values are underlined; % is omitted for each value).

Model	Java					Python				
	R@1	R@5	P@1	P@5	MRR	R@1	R@5	P@1	P@5	MRR
CodeMatcher[47]	53.87	74.52	53.87	49.55	62.78	58.83	79.20	58.83	55.78	68.64
DeepCS[21]	33.22	55.84	33.22	31.71	43.89	53.34	72.61	53.34	51.12	62.83
CARLCS-CNN[57]	41.91	57.19	41.91	38.15	45.11	56.86	77.75	56.86	53.56	66.37
UNIF[8]	52.54	73.18	52.54	47.61	61.25	57.22	78.38	57.22	54.25	66.89
SemEr[14]	56.06	75.45	56.06	52.42	63.54	61.63	81.11	61.63	58.42	69.71
CAT[24]	52.91	73.40	52.91	48.49	61.93	57.90	78.52	57.90	54.83	67.76
TabCS[66]	53.70	74.24	53.70	49.14	62.12	58.37	78.96	58.37	55.22	68.01
GraphSearchNet[49]	56.99	76.03	56.99	53.63	65.80	65.31	84.21	65.31	61.34	73.90
CodeBERT[17]	56.52	75.71	56.52	53.02	65.09	64.75	83.93	64.75	60.57	72.74
GraphCodeBERT(Co-T)	55.34	74.87	55.34	51.46	63.12	60.68	80.59	60.68	58.15	69.22
GraphCodeBERT(Full-T)	56.13	75.52	56.13	52.84	64.23	63.87	82.30	63.87	59.54	71.86
GraphCodeBERT(Co-F)	57.26	77.58	57.26	54.10	66.45	65.80	84.76	65.80	62.19	74.13
GraphCodeBERT(Full-F)	59.89	79.13	59.89	56.32	68.94	66.78	85.82	66.78	64.51	74.97
UniXcoder(Co-F)	58.31	78.46	58.31	55.21	67.80	66.14	85.10	66.14	63.25	74.52
UniXcoder(Full-F)	62.06	80.82	62.06	58.58	70.13	68.25	87.06	68.25	66.27	76.89
CoSrch(ours)	<u>61.32</u>	<u>80.03</u>	<u>61.32</u>	<u>57.70</u>	<u>69.36</u>	<u>67.54</u>	<u>86.21</u>	<u>67.54</u>	<u>65.62</u>	<u>75.95</u>

5.2.2 RQ2 (How do different graph representations in CoSrch impact its performance?) To answer RQ2, CoSrch is compared to three variants.

- **CoSrch(w/o CG):** it is a variant of CoSrch failing to consider the semantic graphs of code snippets (CG for short). The code text in it is processed as a sequence via the multi-head attention instead of the graph encoder.
- **CoSrch(w/o QG):** a variant of CoSrch that does not consider the semantic graphs of queries (QG for short). The query text is processed as a sequence via the multi-head attention instead of the graph encoder.

- **CoSrch(w/o CG&QG)**: a variant of CoSrch which fails to consider the semantic graphs of both queries and code snippets. Similarly, query (or code) texts are processed as a sequence via the multi-head attention.

Table 5 reveals that removing the graphical representation of either code snippets (w/o CG) or queries (w/o QG) leads to a significant performance drop in CoSrch, highlighting the importance of graph-based modeling for both modalities (i.e., query and code snippet) in code search. Furthermore, CoSrch consistently surpasses its variants (w/o CG, w/o QG, and w/o CG&QG), demonstrating the advantage of its unified graph representation for the two modalities. This approach effectively captures cross-modal interactions, such as semantic alignment and structural dependencies, thereby enhancing the quality of code representations in CoSrch.

Table 5. Evaluation of graph representations in CoSrch (w/o means “without” and best values are in bold; % is omitted for each value).

Model	Java					Python				
	R@1	R@5	P@1	P@5	MRR	R@1	R@5	P@1	P@5	MRR
CoSrch(w/o CG)	54.97	75.54	54.97	51.83	64.38	58.21	79.37	58.21	55.12	67.94
CoSrch(w/o QG)	45.17	67.22	45.17	41.94	55.32	53.61	75.53	53.61	51.40	63.62
CoSrch(w/o CG&QG)	44.02	65.01	44.02	39.58	54.20	52.55	74.10	52.55	48.85	62.38
CoSrch(ours)	61.32	80.03	61.32	57.70	69.36	67.54	86.21	67.54	65.62	75.95

5.2.3 *RQ3 (How do the graph encoder and k -hop neighbor graph interaction-based learning affect CoSrch’s performance?)* To address RQ3, CoSrch is compared with six variants of it below.

- **CoSrch(GCN)**: it replaces the graph encoder with a K -layer GCN (Graph Convolutional Network) [36], where K is determined based on CoSrch’s optimal performance, to encode code and query structures.
- **CoSrch(GAT)**: it replaces the graph encoder with a K -layer GAT encoder, to encode code and query structures. Each layer in the encoder has four attention heads.
- **CoSrch(Transformer)**: it replaces the graph encoder with a K -layer transformer encoder. Each layer has four attention heads, following the standard transformer settings.
- **CoSrch(w/o k -hop)**: it removes the k -hop neighbor graph interaction step from CoSrch.
- **CoSrch(hop-MLP)**: in this variant, the GAT in Equation 8 used by the k -hop neighbor graph interaction step is replaced with an MLP.

To evaluate the advantages of the proposed graph encoder in CoSrch, three variants separately built on traditional encoders, namely CoSrch(GCN), CoSrch(GAT), and CoSrch(Transformer) are compared in Table 6, with GCN or GAT’s layer count aligned to CoSrch’s hop count for fair comparison. Results demonstrate CoSrch’s superiority, validating the effectiveness of its graph encoder. This occurs because traditional GNNs (e.g., GCN/GAT) need deep architectures to capture long-range node dependencies - critical for code/query functionality understanding - yet their aggregation paradigm suffers from over-squashing and over-smoothing, often losing these dependencies during message propagation. Transformers, though capable of modeling long-range node relations, often disrupt graph structures in the sequential processing. In contrast, our graph encoder preserves both short- and long-range node dependencies via k -hop neighbor graph interaction-based learning, overcoming these limitations.

To further assess the efficacy of CoSrch’s k -hop neighbor graph interaction-based learning for capturing node dependencies in code (or query) graphs, we evaluated two variants: CoSrch(w/o k -hop), which lacks interaction, and CoSrch(hop-MLP), which employs MLP to learn interactions. Table 6 highlights two critical insights. First,

excluding k -hop neighbor graph interactions, as seen in CoSrch(w/o k -hop), significantly diminishes CoSrch’s performance, with the R@1 score dropping from 61.32 to 55.87 on the Java dataset and from 67.54 to 60.46 on the Python dataset. This underscores the crucial role of learning node dependencies for a deeper understanding of code functionality. Second, when interactions are learned via the MLP in CoSrch(hop-MLP) instead of the GAT used in CoSrch, performance also declines markedly, with the R@1 score decreasing by 7.41% on the Java dataset and by 8.10% on the Python dataset. This is because CoSrch(hop-MLP) relies on the linear concatenation of embeddings from different k -hop neighbor graphs, assigning equal weight to each graph regardless of the length of node dependencies it models. In contrast, CoSrch’s use of a GAT overlay allows it to learn the significance of different k -hop neighbor graphs, thereby maintaining its discriminative power while balancing both short-range and long-range structural patterns in code snippets.

Table 6. Effect of graph encoder and k -hop neighbor graph interaction-based learning (w/o means “without” and best values are in bold; % is omitted for each value).

Model	Java					Python				
	R@1	R@5	P@1	P@5	MRR	R@1	R@5	P@1	P@5	MRR
CoSrch(GCN)	26.65	45.82	26.65	23.77	35.49	31.50	51.47	31.50	28.12	42.74
CoSrch(GAT)	48.10	68.34	48.10	45.20	57.32	53.15	74.18	53.15	51.06	62.39
CoSrch(Transformer)	54.05	74.91	54.05	52.07	62.14	58.76	79.28	58.76	56.14	68.85
CoSrch(w/o k -hop)	55.87	75.29	55.87	53.10	62.89	60.46	80.61	60.46	58.25	69.20
CoSrch(hop-MLP)	57.09	77.42	57.09	54.08	65.45	62.48	82.67	62.48	59.87	71.70
CoSrch(ours)	61.32	80.03	61.32	57.70	69.36	67.54	86.21	67.54	65.62	75.95

5.2.4 RQ4 (How is the generalization ability of CoSrch for different code search scenarios?) Due to space constraints, CoSrch is compared with only five representative baselines to address RQ4, including GraphSearchNet as the top multimodal approach for Java/Python code search and four pretrained multimodal models. Note that GraphCodeBERT (or UniXcoder) labeled as “Full-F” indicates the model retains its original parameter scale and is finetuned on our datasets, whereas baselines labeled “Co-F” adopt the same parameter scale as CoSrch (16.98 MB or 13.20 MB) and are also finetuned on our datasets.

First, to evaluate CoSrch’s capability in handling large-scale codebases containing complex or massive code snippets, we test CoSrch and representative baselines on both C and JavaScript datasets. Table 3 shows that the C dataset contains the largest graphs in scale, averaging 4,037 nodes and 4,676 edges per graph, while Table 2 reveals that the JavaScript dataset has the highest number of code snippets, with 1,051,824 training samples. According to Table 7, CoSrch achieves the best or second-best performance on the C and JavaScript datasets, respectively, demonstrating its strong generalization ability in large-scale code graph and snippet scenarios. Notably, UniXcoder(Full-F) slightly outperforms CoSrch on the JavaScript dataset due to its larger parameter size (135.58 MB) and pretraining on additional high-quality JavaScript samples. Although CoSrch does not lead on all datasets, it surpasses GraphCodeBERT(Full-F), which has more parameters, and significantly outperforms pretrained models with comparable parameter sizes (GraphCodeBERT(Co-F) and UniXcoder(Co-F)). CoSrch’s superiority arises from (1) its effective learning of short- and long-range node dependencies in the graph representations of query and code snippets, and (2) its elimination of redundant information between semantic and syntactic modalities during fusion, which otherwise weakens unique features of single modality (Figure 3). In contrast, the four pretrained Transformer-based models process graphs as linear “variable sequences” or “

flattened AST” for compatibility, disrupting structural relationships. Additionally, their failure to filter redundant inter-modal information further hinders code search performance.

Table 7. Evaluation of CoSrch’s generalization ability (best values are in bold and next-best values are underlined; % is omitted for each value).

Model	C					JavaScript				
	R@1	R@5	P@1	P@5	MRR	R@1	R@5	P@1	P@5	MRR
GraphSearchNet	46.78	67.50	46.78	44.11	56.36	58.75	78.40	58.75	55.83	67.95
GraphCodeBERT(Co-F)	47.39	69.27	47.39	45.65	57.42	60.10	79.84	60.10	57.46	68.70
GraphCodeBERT(Full-F)	51.20	71.36	51.20	48.68	59.24	62.91	81.58	62.91	60.08	71.46
UniXcoder(Co-F)	49.42	70.19	49.42	47.05	58.57	61.45	80.79	61.45	58.50	70.18
UniXcoder(Full-F)	52.57	73.24	52.57	50.34	61.82	65.22	83.63	65.22	61.76	73.34
CoSrch(ours)	53.11	74.08	53.11	51.20	62.25	<u>64.17</u>	<u>82.86</u>	<u>64.17</u>	<u>60.62</u>	<u>72.20</u>

Second, to assess CoSrch’s robustness against over-squashing and over-smoothing issues in processing large-scale code graphs, we evaluate it on the C dataset, which contains the largest scale of code graphs. We compare CoSrch with its GNN-based variants, CoSrch(GCN) and CoSrch(GAT), by varying the maximum layer/hop count K (i.e., the maximum length of node dependencies in CoSrch) considered in the model. As illustrated in Figure 6, CoSrch(GCN) and CoSrch(GAT) suffer a sharp performance drop after reaching their peak, while the CoSrch framework remains stable across deeper layers, demonstrating inherent robustness to both over-squashing and over-smoothing. This confirms CoSrch’s strong robustness across varying hop counts (i.e., varying lengths of node dependencies) in both small- and large-scale graphs, demonstrating the scalability of its graph encoder. Meanwhile, by observing Figure 6, we derive that a significant performance improvement of CoSrch in both R@1 and MRR occurs when K increases from 1 to 8, confirming the importance of long-range node dependencies in capturing code functionality. However, when K exceeds 8, the performance gain of CoSrch diminishes. This may occur because excessively large K values introduce more noise or irrelevant dependencies, weakening the model’s ability to focus on the most critical structural relationships in the code on one hand. On the other hand,

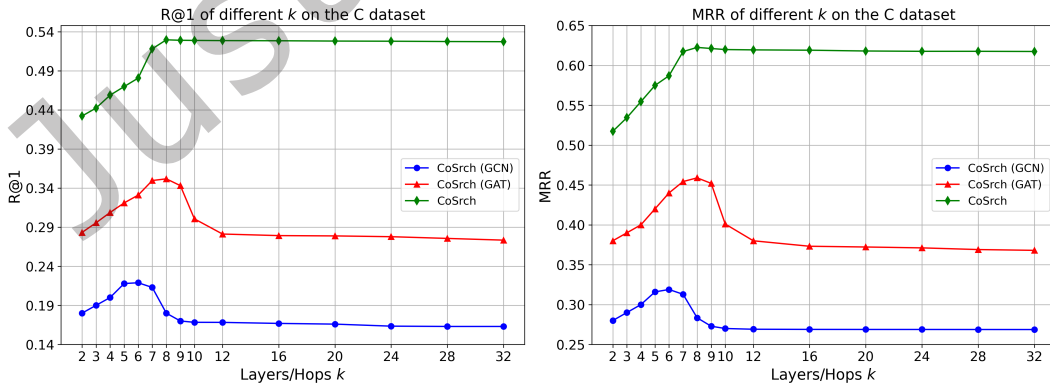


Fig. 6. Comparison of variants of CoSrch by varying K .

important long-range node dependencies that reflect code functionality may become rare in most physical-world code graph structures [11].

5.2.5 RQ5 (How much does OA-MDFF contribute to CoSrch’s effectiveness, and how does its redundancy removal mechanism influence model performance?) Note that OA-MDFF (Overlap-Aware Modality Decomposition and Fusion Framework) in CoSrch is designed to address two key limitations in recent code search studies, namely inadequate capture of long-range dependencies in code structure (Challenge) and ineffective fusion of code syntactic and semantic information. Considering To answer RQ5, the following three variants of CoSrch are evaluated.

- **CoSrch (w/o Syn):** it eliminates inputted code syntactic graphs and the entire OA-MDFF framework, relying solely on a graph encoder to capture the semantic structure of the code snippet. Thus, it retrieves code snippets for a query only based on their semantic similarity.
- **CoSrch (w/o Sem):** it removes inputted code semantic graphs and the OA-MDFF, relying solely on a graph encoder to encode the code syntactic structures. Thus, it retrieves code snippets for a query only based on the code syntactic information.
- **CoSrch (w/o OA-MDFF):** it removes the OA-MDFF from CoSrch, while a basic fusion method [66] is used to integrate code syntactic and semantic structure information. Note that the proposed graph encoders are still applied to encode both syntactic and semantic structures of codes in the variant.

Table 8. Effect of OA-MDFF in CoSrch (w/o means “without” and best values are in bold; % is omitted for each value).

Model	Java					Python				
	R@1	R@5	P@1	P@5	MRR	R@1	R@5	P@1	P@5	MRR
CoSrch(w/o Syn)	52.27	73.11	52.27	50.11	61.79	57.30	78.82	57.30	55.25	67.15
CoSrch(w/o Sem)	52.89	73.53	52.89	50.60	62.23	58.65	79.14	58.65	56.24	67.81
CoSrch(w/o OA-MDFF)	53.34	74.30	53.34	51.45	62.82	59.33	80.55	59.33	57.16	69.08
CoSrch(ours)	61.32	80.03	61.32	57.70	69.36	67.54	86.21	67.54	65.62	75.95

The ablation study results of OA-MDFF on Java and Python datasets are listed in Table 8. The table shows that CoSrch performs significantly better than the three variants. Specifically, the MRR of CoSrch is on average 10.18% higher than that of the best variant, i.e., CoSrch(w/o OA-MDFF), on both Java and Python datasets. It is worth noting that CoSrch(w/o OA-MDFF) also considers the two modalities (syntactic and semantic) in code search but uses a naive fusion strategy to integrate them. The superior performance of CoSrch stems from its effective handling of redundant information between the two modalities and its emphasis on the unique characteristics of each single modality. Additionally, Table 8 demonstrates that CoSrch(w/o OA-MDFF) outperforms the other two variants, which either ignore syntactic structure or disregard semantic structure of a code snippet. This finding further confirms the importance of incorporating both modalities (syntactic and semantic) in code representation.

5.2.6 RQ6 (How do three types of loss functions ($\mathcal{L}_{de}$, $\mathcal{L}$, and $\mathcal{L}_{\Delta}$) in CoSrch impact its performance?) Restate that $\mathcal{L}_{de}$ represents the multimodal decomposition loss in Equation 12, which decomposes the non-overlapping and overlapping information between two modalities. $\mathcal{L}$ in Equation 13 (or $\mathcal{L}_{\Delta}$ in Equation 14) denotes the semantic (or syntactic) reconstruction loss, incurred when reconstructing each modal representation from the non-overlapping and overlapping information of the two modalities derived by $\mathcal{L}_{de}$. This ensures effective separation of the two information types within each modal representation. To address RQ6, five variants of CoSrch are evaluated, each

of which is designed by varying the weighting parameters (α , β , and γ) of those three loss functions in CoSrch’s overall loss function (Equation 22). Experimental results are demonstrated in Table 9.

- **CoSrch(w/o $\mathcal{L}_{de}$ & $\mathcal{L}$ & $\mathcal{L}_\Delta$)**: it in fact equals to CoSrch(w/o OA-MDFF), which disables multimodal decomposition in CoSrch.
- **CoSrch(w/o $\mathcal{L}$ & $\mathcal{L}_\Delta$)**: it removes $\mathcal{L}$ and $\mathcal{L}_\Delta$ from the overall loss of CoSrch (Equation 22) and set α to 1.
- **CoSrch(w/o $\mathcal{L}_\Delta$)**: it deletes $\mathcal{L}_\Delta$ from the overall loss, while α and β are both set to 1.
- **CoSrch(w/o $\mathcal{L}$)**: it eliminates $\mathcal{L}$ from the overall loss, while α and γ are both set to 1.
- **CoSrch(eql-wt)**: it fixes all weighting parameters (α , β , and γ) to 1 in the overall loss function.

Table 9. Effect of different fusion functions on CoSrch’s performance (w/o means “without” and best values are in bold; % is omitted for each value).

Model	Java					Python				
	R@1	R@5	P@1	P@5	MRR	R@1	R@5	P@1	P@5	MRR
CoSrch(w/o $\mathcal{L}_{de}$ & $\mathcal{L}$ & $\mathcal{L}_\Delta$)	53.44	74.30	53.34	51.45	62.82	59.33	80.55	59.33	57.16	69.08
CoSrch(w/o $\mathcal{L}$ & $\mathcal{L}_\Delta$)	58.44	77.90	58.44	56.11	67.84	66.35	85.20	66.35	64.15	74.91
CoSrch(w/o $\mathcal{L}_\Delta$)	60.46	78.95	60.46	57.00	68.72	67.40	85.94	67.40	65.33	75.69
CoSrch(w/o $\mathcal{L}$)	60.80	79.48	60.80	57.56	69.15	66.91	85.48	66.91	65.08	75.16
CoSrch(eql-wt)	60.68	79.21	60.68	57.29	68.97	67.16	85.75	67.16	65.20	75.38
CoSrch(ours)	61.32	80.03	61.32	57.70	69.36	67.54	86.21	67.54	65.62	75.95

Table 9 first demonstrates that CoSrch, which concurrently applies all three loss types ($\mathcal{L}_{de}$, $\mathcal{L}$, and $\mathcal{L}_\Delta$) for code representation learning, achieves the best performance compared to its five variants on both Java and Python datasets. Second, CoSrch(w/o $\mathcal{L}_{de}$ & $\mathcal{L}$ & $\mathcal{L}_\Delta$), which fails to decompose overlapping and non-overlapping inter-modal information during fusion, exhibits the worst code search performance (R@1 averagely drops by 14.30% versus CoSrch). This confirms the necessity of eliminating redundant inter-modal information in code search tasks. Third, among all variants, CoSrch(w/o $\mathcal{L}$ & $\mathcal{L}_\Delta$) ranks second-worst, highlighting the critical role of semantic/syntactic reconstruction losses in optimizing the multimodal decomposition loss $\mathcal{L}_{de}$, as they iteratively expand the overlapping and non-overlapping information within inter-modal representations. Table 9 also reveals that learnable loss weighting (CoSrch) outperforms fixed equal weighting (CoSrch(eql-wt)), underscoring its importance for high-quality code representations. Notably, CoSrch(w/o $\mathcal{L}_\Delta$) performs better on Python but worse on Java than CoSrch(w/o $\mathcal{L}$). This discrepancy arises because Java’s more complex syntactic graphs (Table 3) benefit more from retaining syntactic reconstruction, whereas Python’s runtime semantics—driven by its flexibility and interpreted nature—prioritize semantic preservation ($\mathcal{L}$). Finally, Table 9 shows that CoSrch(eql-wt) sometimes is beaten by CoSrch(w/o $\mathcal{L}_\Delta$) or CoSrch(w/o $\mathcal{L}$). This occurs because fixed equal weighting fails to account for language-specific characteristics: for Java, equally weighted losses undervalue syntactic importance ($\mathcal{L}_\Delta$), while for Python, they undervalue semantic importance ($\mathcal{L}$). These results reinforce the significance of adaptive weighting for $\mathcal{L}_{de}$, $\mathcal{L}$, and $\mathcal{L}_\Delta$ in CoSrch’s total loss.

5.2.7 RQ7 (What are CoSrch’s training and inference efficiency?) To answer RQ7, Tables 10 and 11 compare the training time, inference time, and trainable parameters of different models on JavaScript and C datasets, which respectively contain the largest-scale code snippets and code graphs. Notably, due to vocabulary size differences between JavaScript and C, the parameter size for the same method varies across datasets. The tables reveal

that training/inference time is generally proportional to parameter size, with larger models incurring higher computational costs. However, GraphSearchNet exhibits an anomaly: despite having the smallest parameter size, it consumes more computation time due to its complex graph construction and BiGGNN with multi-head attention modules that capture local/global dependencies at the expense of increased overhead. Importantly, our proposed CoSrch demonstrates superior efficiency in both training and inference across both datasets, even when processing large-scale code graphs. In specific, CoSrch inherently addresses scalability through several key mechanisms: (1) the k -hop neighbor graph interaction approach avoids explicit full-graph computations; (2) it is empirically reflected by Figure 6 that limiting the maximum length of node dependency path (i.e., K) to 8 (equivalent to computing adjacency matrix powers up to A^8) achieves optimal effectiveness-complexity trade-offs; (3) the overlap-aware decomposition reduces redundant computations by focusing on distinct features of every modal (syntactic or semantic). We are delighted to observe from Table 11 that CoSrch’s advantage is particularly evident when contrasting with CoSrch(GAT) which requires 4.0% more training time and 16.42% more inference time on the C dataset having the largest-scale of code graphs due to its multi-layer computations for learning long-range node dependencies in code graphs.

Table 10. Comparison of model efficiency on JavaScript dataset. (M denotes "million".)

Model	Training time	Inference time	Training Parameters
GraphSearchNet	8.8 hours	1.10 s/query	10.91 M
CoSrch(GAT)	8.3 hours	1.03 s/query	15.36 M
GraphCodeBERT(Co-F)	9.4 hours	1.12 s/query	16.89 M
GraphCodeBERT(Full-F)	11.2 hours	3.14 s/query	125.00 M
UniXCoder(Co-F)	10.1 hours	1.20 s/query	16.89 M
UniXCoder(Full-F)	12.3 hours	3.35 s/query	135.58 M
CoSrch(ours)	7.8 hours	0.92 s/query	16.89 M

Table 11. Comparison of model efficiency on C dataset. (M denotes "million".)

Model	Training time	Inference time	Training Parameters
GraphSearchNet	5.6 hours	0.74 s/query	5.88 M
CoSrch(GAT)	5.2 hours	0.78 s/query	12.45 M
GraphCodeBERT(Co-F)	6.0 hours	0.81 s/query	13.20 M
GraphCodeBERT(Full-F)	7.1 hours	2.85 s/query	125.68 M
UniXCoder(Co-F)	6.4 hours	0.87 s/query	13.20 M
UniXCoder(Full-F)	8.2 hours	3.08 s/query	136.26 M
CoSrch(ours)	5.0 hours	0.67 s/query	13.20 M

5.2.8 RQ8 (How effective is CoSrch in handling real-world queries compared to baselines?) CoSrch has demonstrated advantageous performance in aforementioned automatic evaluation experiments. However, in practice, human programmers are best suited to judge the usefulness of a returned code snippet. Therefore, we conduct a user study with three experienced developers to evaluate the usefulness of code snippets retrieved by representative baselines (GraphSearchNet, GraphCodeBERT (Full-T), UniXCoder (Co-F)) and our proposal (CoSrch), based on real-world queries.

We develop a search platform based on a codebase containing 756,000 Java and Python functions, as well as 99 real-world queries sourced from CodeSearchNet⁵. These queries are derived from common *Bing* search terms with high click-through rates for code-related results and included clear technical keywords. Three developers involved in the evaluation are blinded to the source model of each retrieved result. For each query, every model retrieves the top-10 matching code snippets. Developers then are asked to manually evaluate the usefulness of each retrieved snippet, resolving any disagreements through discussion. The experimental results, summarized in Table 12, report average scores assigned by three developers across 99 queries. As shown in the table, CoSrch achieves an average R@10 of 82.56% for Java and 88.24% for Python, significantly outperforming all three baselines. Additionally, CoSrch’s MRR is 1.6% and 1.3% higher than the best-performing baseline, GraphCodeBERT(Full-T). These results demonstrate CoSrch’s effectiveness in addressing physical-world queries and retrieving highly useful code snippets for developers. Its superior performance over baseline methods underscores its practical value in real-world code search scenarios.

Table 12. Effectiveness of different models in handling real-world queries (best values are in bold; % is omitted for each value).

Model	Java			Python		
	R@10	P@10	MRR	R@10	P@10	MRR
GraphSearchNet	79.32	52.56	62.97	85.63	57.08	69.80
GraphCodeBERT(Full-T)	81.48	54.22	65.26	87.41	59.15	71.42
UniXcoder(Co-F)	80.45	53.41	64.40	86.55	57.91	70.68
CoSrch(ours)	82.56	55.30	66.32	88.24	60.30	72.35

To further validate our approach’s effectiveness on real-world queries, Tables 13 and 14 compare the top-1 Python code snippets retrieved by our CoSrch model against two strong code search baselines from Table 4: GraphSearchNet (a multimodal model) and GraphCodeBERT (Full-F) (a pre-trained multimodal model). The queries used for Tables 13 and 14 are respectively “hash set for counting distinct elements” and “convert int to string”. Both tables include the ground truth for their concerned queries alongside the query-code similarity scores computed based on different code search models. Notably, the ground truth is absent from the codebase in Table 13, but present in the codebase in Table 14.

As shown by Tables 13 and 14, CoSrch’s top-1 code snippet consistently demonstrates superior alignment with the query. As shown in Table 13, only CoSrch’s top-1 retrieved result fully satisfies the query requirement—counting distinct elements in a hash set. In contrast, GraphSearchNet’s top-1 snippet incorrectly computes the sum of distinct values, while GraphCodeBERT(Full-F) mistakenly checks for set membership, both fundamentally deviating from the intended task of the query. The effectiveness of CoSrch stems from two key innovations. First, it precisely models both short-range dependencies (e.g., local logical relationships like between a “for” loop and its nested “if” condition) and long-range dependencies (e.g., variable initialization “count=0” and its subsequent update “count+=1”), thereby capturing the complete functional logic of a code snippet. Second, its multimodal fusion mechanism systematically eliminates redundant information between semantic and syntactic representations, allowing CoSrch to focus on critical nodes (e.g., the increment value “1” in “count+=1”). In contrast, GraphSearchNet directly integrates a code snippet’s sequential code text with its AST structure, which retains redundant node information, causing it to miss key signals like the increment value “item” in “total+=item”. Similarly, GraphCodeBERT(Full-F) fails to adequately represent local variable-level logic (e.g., between “target” and “tracker”), since it processes graphs as linear “variable sequences” for compatibility and as a result disrupts

⁵<https://github.com/github/CodeSearchNet/blob/master/resources/queries.csv>

local structural relationships. As indicated by the fourth column of Table 13, the most relevant code snippet retrieved by GraphSearchNet or GraphCodeBERT(Full-F) gains relatively low query-code similarity score based on the computed representations by CoSrch (i.e., S_{CoSrch}). Notably, even when the ground truth code snippet is absent from the codebase in Table 13, we observe that its S_{CoSrch} value significantly exceeds that of CoSrch’s top-1 retrieved result, attributable to its clearer semantics and conciseness. As to Table 14, where the ground truth code snippet exists in the codebase, CoSrch accurately retrieves it as the top-1 result. However, for the same query requiring int-to-string conversion, GraphSearchNet incorrectly converts a value of uncertain type to float, and GraphCodeBERT(Full-F) fails to verify whether the input data is of type int. Both approaches ultimately fail to meet the query requirements. GraphSearchNet fails because it overlooks redundant information between the text modality and AST structure modality of a code snippet, diluting the critical signal from the “float” node. Meanwhile, GraphCodeBERT misaligns with the query intent due to inadequate local logic modeling in “value=int(value)”. In contrast, CoSrch effectively eliminates modality redundancy during integration, properly attending to “int” in “value=int(value)” and thus ensuring the converted value must be an integer. Meanwhile, based on the CoSrch-learned representations of queries and code snippets, the similarity score of the top-1 result retrieved by GraphCodeBERT (Full-F) is significantly higher than that of GraphSearchNet’s top-1 result, which has the lowest relevance to the query. This comparison once again verifies the high quality of the learned representations in our proposal.

Finally, we present a running example in Table 15 where CoSrch fails to retrieve the correct code snippet for the query “encode url” on Python dataset. As shown in the table, GraphCodeBERT outperforms both GraphSearchNet and CoSrch by returning an exact match, while GraphSearchNet retrieves an incorrect URL decoding code snippet and CoSrch returns an irrelevant result. This discrepancy occurs because GraphSearchNet and CoSrch depend on dependency parsing to construct token-relationship graphs, which proves insufficient for very short queries like “encode url”. The overly simplistic graph structure lacks the necessary semantic information, causing both models to perform poorly in code search tasks. This example underscores the limitations of graph-based methods when handling extremely short queries.

Table 13. Running examples of the top-1 code snippet retrieved by different models on Python dataset for the query “hash set for counting distinct elements”. (S_* represents the cosine similarity between a query and its retrieved code snippet, computed over their respective embeddings generated by model *)

GraphSearchNet	GraphCodeBERT (Full-F)	CoSrch	Ground_Truth
<pre>def sum_uniq(items): tracker = {} total = 0 for item in items: if item not in tracker: tracker[item] = True total += item return total</pre>	<pre>def check(items, target): tracker = {} for item in items: tracker[item] = True exist = target in tracker return exist</pre>	<pre>def count_uniq(items): tracker = {} count = 0 for item in items: if item not in tracker: tracker[item] = True count += 1 return count</pre>	<pre>def count_distinct(items): unique_items = set() for item in items: unique_items.add(item) return len(unique_items)</pre>
$S_{GraphSearchNet}=0.7350$ $S_{CoSrch}=0.6796$	$S_{GraphCodeBERT}=0.6948$ $S_{CoSrch}=0.6279$	$S_{CoSrch}=0.8074$	$S_{CoSrch}=0.8123$

5.3 Threats to Validity

The internal threats to validity originate primarily from our implementation design.

First, the effective separation of overlapping and non-overlapping features of two modalities is a non-trivial task. Since the design of loss function for separation, training dynamics, and noises contained in datasets all

Table 14. Running examples of the top-1 code snippet retrieved by different models on Python dataset for the query “convert int to string”. (S_* represents the cosine similarity between a query and its retrieved code snippet, computed over their respective embeddings generated by model *)

GraphSearchNet	GraphCodeBERT (Full-F)	CoSrch	Ground_Truth
def to_string(value): value = float(value) return str(value)	def number_transform (value): return str(value)	def to_string(value): value = int(value) return str(value)	def to_string(value): value = int(value) return str(value)
$S_{GraphSearchNet}=0.7480$ $S_{CoSrch}=0.6654$	$S_{GraphCodeBERT}=0.8072$ $S_{CoSrch}=0.7871$	$S_{CoSrch}=0.8345$	$S_{CoSrch}=0.8345$

Table 15. Running examples of results retried from Python dataset by different models for the query “encode url”.

GraphSearchNet	GraphCodeBERT (Full-F)	CoSrch
def decode_url(encoded_string): return urllib.parse. unquote(encoded_string)	def encode_url(input_string): return urllib.parse. quote(input_string)	def reverse_string(s): return s[::-1]

have impact on the separation effectiveness. In this case, to improve the separated results of overlapping and non-overlapping features, we not only designs the decoupling loss function $\mathcal{L}_{de}$ for feature decomposition, but also proposes two reconstruction loss functions ($\mathcal{L}$ and $\mathcal{L}_{\Delta}$) to iteratively expand those share parts and distinct parts of two modalities in the encoded representation of every modal, enhancing CoSrch’s robustness against noisy real-world data.

Second, constructing the syntactic graph for a code snippet in CoSrch is highly challenging when the snippet lacks complete dependency relationships among nodes, such as missing the function name, which are crucial for ensuring CoSrch’s effectiveness in understanding code functionality. However, in community-contributed code bases like Stack Overflow, syntactic graphs of code snippets may have incomplete node dependencies, undermining CoSrch’s effectiveness. A potential solution is to automatically generate minimal interfaces for those missing classes or methods in snippets, which we leave as future work.

Third, variations in baseline implementations and hardware limitations may influence experimental outcomes, particularly due to the sensitivity of deep learning models to hyperparameter choices such as learning rate and embedding dimensionality, which could affect comparative evaluations. To ensure consistency and reproducibility of models to be compared, we employed widely recognized open-source baselines [8, 13, 14, 17, 21, 23, 24, 49, 57, 66] using their originally reported configurations, thereby maintaining alignment with prior works.

Finally, another critical threat to validity stems from treating functional correctness of code snippets as an external constraint rather than an intrinsic retrieval criterion [45, 64]. This limitation is inherent not only in our approach but also in existing code retrieval systems [14, 49, 67], where semantically similar code fragments may contain subtle functional defects such as improper edge case handling or concurrency issues. These cases directly lead to precision degradation in retrieval code snippets. To address this limitation, we plan to investigate hybrid approaches that augment retrieval with lightweight dynamic analysis techniques like execution trace matching, to improve functional correctness assurance in future work.

5.4 Discussion

This section systematically analyzes the technical integration mechanisms of the proposed CoSrch model within developer workflows and explores its advantages in the era of generative artificial intelligence.

5.4.1 Analysis of Technical Integration Pathways. The CoSrch model exhibits a high degree of compatibility with existing development tools due to its seamless integration with widely used platforms. By embedding ElasticSearch as a plugin within mainstream integrated development environments (IDEs), including the JetBrains suite and AI-assisted tools like GitHub Copilot, CoSrch enables users to perform natural language queries for retrieving relevant code snippets from code bases. The process begins when development tools invoke CoSrch's query semantic encoder to transform user queries into vector representations. These vectors are then used to search a pre-vectorized code base stored in ElasticSearch, where code snippets have been encoded offline using CoSrch's OA-MDFF model. This architecture ensures efficient and accurate code search functionality. For example, preprocessing 756k Java or Python code snippets into vectors offline requires approximately 70 minutes, but querying via ElasticSearch delivers 10 candidate results in about 0.25 seconds, fulfilling real-time performance demands.

5.4.2 Advantages Compared to Recent Generative AI-based Code Generation/search Solutions. CoSrch outperforms recent generative AI-based code generation or search systems (e.g., Codex [12]) by directly tackling their core weaknesses: unreliable outputs and inefficient code structural understanding. While generative models frequently hallucinate incorrect or irrelevant code due to their probabilistic nature, CoSrch guarantees accuracy by leveraging a graph-based architecture that explicitly models long-range dependencies and syntactic-semantic relationships via dedicated graph encoders. Its overlap-aware modality decomposition framework further enhances precision by intelligently fusing syntax and semantics while filtering redundant data, resulting in more interpretable and contextually grounded matches. Unlike generative AI's opaque decision-making, CoSrch operates on transparent code representations, ensuring retrievals are both fast and verifiable. This makes it more suitable for real-world development where correctness and efficiency matter more than speculative generation. By focusing on retrieval over synthesis, CoSrch avoids the pitfalls of unvetted AI outputs while maintaining the adaptability needed for diverse codebases, striking an optimal balance between performance and reliability that purely generative approaches cannot match. By prioritizing precise retrieval, CoSrch avoids the risks of unverified AI-generated outputs while retaining generalization capability across diverse codebases, achieving an optimal balance of speed and accuracy that purely generative methods lack.

6 Related Works

6.1 Code Representation

Initially, research efforts on code representation primarily focused on token sequences [2, 21, 30]. The token-based approach to code representation involves segmenting code snippets into sequences of tokens, with each token being mapped to a real-valued vector. However, this approach tends to disregard the syntactic structure information inherent in the code. Another technological route is to model the syntactic structure of codes using the abstract syntax tree (AST) structure [53]. Notable works in this area include Tree-CNN [53], Tree-LSTM [60], Code2vec [4], and DeepCom [28]. All these works propose to represent the AST structure by linearizing it into a sequence of nodes using classical tree traversal strategies. Converting an AST structure to a node sequence will compromise the 2D structural integrity of the AST structure. Thus, recent advancements in code representation have seen the proliferation of graph-based approaches [1] for learning semantics of codes across various tasks, including source code summarization [18, 52] and code search [44, 69]. As an example, MMF3 [52] integrates the AST syntactic structure encoded by a GNN (named GCN [36]) with the semantic information extracted from the code sequences via transformers, and as a result successfully enhance the performance of source code summarization. Although GNN can extract the short-range dependencies among nodes in ASTs, it can rarely capture the long-range dependencies among nodes in ASTs. Seeing this limitation, Gao and Lyu introduce a multi-scale feature extraction method named M2TS [18]. M2TS can effectively capture diverse short&long-range dependencies among nodes in the syntactic structure graph of a code snippet by considering multiple subgraphs

with different scales. The computational burden of M2TS, however, raises very quickly with the increase of the graph scale, due to the training of graph convolutional network for each subgraph. Moreover, there exists constrained interaction among distinct subgraphs, hindering the comprehensive utilization of inter-subgraph relationship information. Hence, it is imperative to investigate a more effective and efficient approach to extract and then make use of the short&long-range dependencies among nodes in the syntactic structure graph of codes for better code representation.

6.2 Code Search

Initially, code search approaches [33, 41, 51] primarily rely on technologies such as information retrieval (IR) and natural language processing (NLP), which both consider the textual similarity between code snippets and queries. Specifically, to improve the similarity comparison between codes and queries, Lu et al. [50] propose to expand queries with their synonyms generated from WordNet and finally successfully enhance the hit ratio of the code search task. CodeHow [51] provides another way of query expansion to improve the code search performance. It first searches a code API repository based on each query, after which all retrieved APIs are used as the supplementary information of the query. CodeMatcher [47], on the other hand, Optimizing the code search procedure involves filtering out irrelevant or noisy keywords in a query or code snippet. However, all the aforementioned approaches retrieve code snippets based solely on textual similarity between the query and code snippet, thereby ignoring the more critical semantic-level similarity for effective query-code matching.

With the advancement of NLP and deep learning, numerous works [8, 43, 56] have emerged which employ deep neural networks to optimize the semantic representations of both queries and code snippets. DeepCS [21] is the pioneering endeavor to use DNNs for code search tasks. Specifically, DeepCS employs multiple LSTMs to encode code API sequences, code tokens, and function names, generating their vector representations. After that, to alleviate the computational complexity of DeepCS, UNIF [8] introduces a straightforward bag-of-word model with an attention mechanism, demonstrating enhanced performance of code search tasks. CARLCS-CNN [57] and SAN-CS [16], on the other hand, leverage co-attention mechanisms [6] and self-attention networks respectively, to achieve the semantic mapping from user queries to code snippets. While these models excel in capturing semantic information between queries and code snippets, they neglect the semantic or syntactic structure information in every code snippet, leading to less satisfactory searching results. Seeing this limitation, DGMS [44] proposes to model the semantic or syntactic structure information in code snippets via the AST structure, which largely preserves the structure information in them.

In recent years, numerous studies have aimed to boost code search performance by fusing multi-modality information within code snippets. For example, Haldar et al. [24] introduce CAT, which achieve effective code search by enriching code representations via integrating the modal of code texts and the modal of AST syntactic structures extracted from codes. Meanwhile, MMAN [63] presents a multi-modal attention fusion layer that assigns weights to various segments within each modality of a code snippet, and those segments are then consolidated into a unified hybrid representation of code snippet which captures the implicit semantic relations between the code text and AST structure of the code snippet. MMAN [63], however, focuses solely on inter-modality attention, overlooking the significance of intra-modality information [13]. TabCS [66] is another example. For a certain code snippet, TabCS not only extracts its textual features (i.e., method name, API sequence, and tokens), but also derives its syntactic structure features in the form of AST at first. After that, a two-stage attention network structure employed in TabCS to learn interactions within a single feature modal or between two feature modalities. Finally, Unixcoder [13] introduces a pre-trained multi-modal approach designed to comprehensively explore the relationships between diverse modalities, such as code text and AST structure of code. However, these methods primarily concentrate on maximizing the shared information across all modalities, while overlooking the exploration of distinctive information unique to each modality. In code search, the syntactic and semantic

structures of code snippets contain rich information that requires careful processing and effective fusion. However, existing methods struggle to optimally fuse these modalities. To address this, we propose a novel decoupling fusion method that reduces redundancy when integrating syntactic and semantic structures.

7 Conclusions and Future Works

Seeing the two challenges that are not well addressed in recent code search literature, this paper introduces CoSrch, a novel syntactic and semantic structure-aware code representation deep model for effective code search. Different from related works, CoSrch proposes to model both of the syntactic and semantic structure information in a code snippet uniformly using graphs, to facilitate the alignment of the two types of structure information when fusing them. Then, an effective encoding strategy is designed in CoSrch which can compute a better representation for code (or query) based on adjacency matrices of a syntactic (or semantic) graph with different orders, so as to ensure that long-range dependencies among nodes in the graph can be learned in those representations (addressing Challenge 1). Moreover, we present a overlap-aware modality decomposition and fusion framework in CoSrch, which eliminates the negative impact brought by the redundant information between the two modalities of a code snippet, i.e., syntactic structure and semantic structure, on the quality of the fused code representation based on the two modalities (addressing Challenge 2). Extensive experiments conducted on Java and Python datasets demonstrate the superiority of our proposal over all baselines.

Notice that the proposed CoSrch is a general deep network for representing codes written in different programming languages. In the future, we plan to develop specific programming language-oriented optimization strategies which consider characteristics of different programming languages.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Nos. 62572135, 62272093, U2436208, and 62372129), Guangdong Province Basic and Applied Basic Research Fund Project (Nos. 2025A1515011582 and 2025A1515012952), Guangdong S&T Program (No. 2024B0101010002), and the Project of Guangdong Key Laboratory of Industrial Control System Security (No. 2024B1212020010).

References

- [1] Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. 2018. Learning to Represent Programs with Graphs. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=BJOFETxR->
- [2] Miltiadis Allamanis, Hao Peng, and Charles Sutton. 2016. A Convolutional Attention Network for Extreme Summarization of Source Code. In *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016 (JMLR Workshop and Conference Proceedings, Vol. 48)*, Maria-Florina Balcan and Kilian Q. Weinberger (Eds.). JMLR.org, 2091–2100. <http://proceedings.mlr.press/v48/allamanis16.html>
- [3] Uri Alon and Eran Yahav. 2020. On the Bottleneck of Graph Neural Networks and its Practical Implications. *ArXiv abs/2006.05205* (2020). <https://api.semanticscholar.org/CorpusID:219558760>
- [4] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. 2019. code2vec: learning distributed representations of code. *Proc. ACM Program. Lang.* 3, POPL, Article 40 (jan 2019), 29 pages. doi:10.1145/3290353
- [5] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. *ArXiv preprint abs/1607.06450* (2016). <https://arxiv.org/abs/1607.06450>
- [6] Parisa Bazmi, Masoud Asadpour, and Azadeh Shakery. 2023. Multi-view co-attention network for fake news detection by modeling topic-specific user and news source credibility. *Inf. Process. Manage.* 60, 1 (jan 2023), 17 pages. doi:10.1016/j.ipm.2022.103146
- [7] Tal Ben-Nun, Alice Shoshana Jakobovits, and Torsten Hoefler. 2018. Neural code comprehension: a learnable representation of code semantics (*NIPS'18*). Curran Associates Inc., Red Hook, NY, USA, 3589–3601.
- [8] Jose Cambronero, Hongyu Li, Seohyun Kim, Koushik Sen, and Satish Chandra. 2019. When deep learning met code search. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 964–974.

- [9] Brock Angus Campbell and Christoph Treude. 2017. NLP2Code: Code snippet content assist via natural language tasks. In *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 628–632.
- [10] Casey Casalnuovo, Kenji Sagae, and Prem Devanbu. 2019. Studying the difference between natural and programming language corpora. *Empirical Softw. Engg.* 24, 4 (aug 2019), 1823–1868. doi:10.1007/s10664-018-9669-7
- [11] Jie Chen, Zilong Li, Yin Zhu, Junping Zhang, and Jian Pu. 2023. From node interaction to hop interaction: New effective and scalable graph learning paradigm. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 7876–7885.
- [12] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [13] Junyan Cheng, Iordanis Fostiropoulos, and Barry Boehm. 2020. GN-Transformer: Fusing AST and Source Code information in Graph Networks. (2020).
- [14] Zhongyang Deng, Ling Xu, Chao Liu, Luwen Huangfu, and Meng Yan. 2024. Code semantic enrichment for deep code search. *Journal of Systems and Software* 207 (2024), 111856.
- [15] Lun Du, Xiaozhou Shi, Yanlin Wang, Ensheng Shi, Shi Han, and Dongmei Zhang. 2021. Is a Single Model Enough? MuCoS: A Multi-Model Ensemble Learning Approach for Semantic Code Search. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management (Virtual Event, Queensland, Australia) (CIKM '21)*. Association for Computing Machinery, New York, NY, USA, 2994–2998. doi:10.1145/3459637.3482127
- [16] Sen Fang, You-Shuai Tan, Tao Zhang, and Yepang Liu. 2021. Self-Attention Networks for Code Search. *Information and Software Technology* 134 (2021), 106542. doi:10.1016/j.infsof.2021.106542
- [17] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139
- [18] Yuexiu Gao and Chen Lyu. 2022. M2ts: Multi-scale multi-modal approach based on transformer for source code summarization. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*. 24–35.
- [19] Jian Gu, Zimin Chen, and Martin Monperrus. 2021. Multimodal Representation for Neural Code Search. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 483–494. doi:10.1109/ICSME52107.2021.00049
- [20] Wenchao Gu, Zongjie Li, Cuiyun Gao, Chaozheng Wang, Hongyu Zhang, Zenglin Xu, and Michael R. Lyu. 2021. CRaDL: Deep code retrieval based on semantic Dependency Learning. *Neural Netw.* 141, C (sep 2021), 385–394. doi:10.1016/j.neunet.2021.04.019
- [21] Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. 2018. Deep code search. In *Proceedings of the 40th International Conference on Software Engineering*. 933–944.
- [22] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 7212–7225. doi:10.18653/v1/2022.acl-long.499
- [23] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin B. Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. <https://openreview.net/forum?id=jLoC4ez43PZ>
- [24] Rajarshi Haldar, Lingfei Wu, JinJun Xiong, and Julia Hockenmaier. 2020. A Multi-Perspective Architecture for Semantic Code Search. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 8563–8568. doi:10.18653/v1/2020.acl-main.758
- [25] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. doi:10.1109/CVPR.2016.90
- [26] Abram Hindle, Earl T. Barr, Zhendong Su, Mark Gabel, and Premkumar Devanbu. 2012. On the naturalness of software. In *2012 34th International Conference on Software Engineering (ICSE)*. 837–847. doi:10.1109/ICSE.2012.6227135
- [27] Fan Hu, Yanlin Wang, Lun Du, Xirong Li, Hongyu Zhang, Shi Han, and Dongmei Zhang. 2023. Revisiting code search in a two-stage paradigm. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*. 994–1002.
- [28] Xing Hu, Ge Li, Xin Xia, David Lo, and Zhi Jin. 2018. Deep code comment generation. In *Proceedings of the 26th Conference on Program Comprehension (Gothenburg, Sweden) (ICPC '18)*. Association for Computing Machinery, New York, NY, USA, 200–210. doi:10.1145/3196321.3196334
- [29] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *ArXiv preprint abs/1909.09436* (2019). <https://arxiv.org/abs/1909.09436>
- [30] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2016. Summarizing Source Code using a Neural Attention Model. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Katrin Erk and Noah A. Smith (Eds.). Association for Computational Linguistics, Berlin, Germany, 2073–2083. doi:10.18653/v1/P16-1195

- [31] XU Jia, QIAN Weiwei, YU Ge, and LÜ Pin. 2023. GameOJ: A Programming Ability-aware Gamification Online Judge System. *Journal of South China Normal University (Natural Science Edition)* 55, 1 (2023), 65–77.
- [32] Daniel Jurafsky and James H. Martin. 2000. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition* (1st ed.). Prentice Hall PTR, USA.
- [33] Iman Keivanloo, Juergen Rilling, and Ying Zou. 2014. Spotting working code examples. In *Proceedings of the 36th International Conference on Software Engineering*. 664–675.
- [34] Alex Kendall, Yarin Gal, and Roberto Cipolla. 2018. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 7482–7491.
- [35] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). <http://arxiv.org/abs/1412.6980>
- [36] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=SJU4ayYgl>
- [37] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, et al. 2022. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533* (2022).
- [38] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2017. ImageNet classification with deep convolutional neural networks. *Commun. ACM* 60, 6 (may 2017), 84–90. doi:10.1145/3065386
- [39] C. Lattner and V. Adve. 2004. LLVM: a compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004*. 75–86. doi:10.1109/CGO.2004.1281665
- [40] Qimai Li, Zhichao Han, and Xiao-Ming Wu. 2018. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence* (New Orleans, Louisiana, USA) (AAAI’18/IAAI’18/EAAI’18). AAAI Press, Article 433, 8 pages.
- [41] Xuan Li, Zerui Wang, Qianxiang Wang, Shoumeng Yan, Tao Xie, and Hong Mei. 2016. Relationship-aware code search for JavaScript frameworks. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 690–701.
- [42] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard S. Zemel. 2016. Gated Graph Sequence Neural Networks. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). <http://arxiv.org/abs/1511.05493>
- [43] Chunyang Ling, Zeqi Lin, Yanzhen Zou, and Bing Xie. 2020. Adaptive deep code search. In *Proceedings of the 28th International Conference on Program Comprehension*. 48–59.
- [44] Xiang Ling, Lingfei Wu, Saizhuo Wang, Gaoning Pan, Tengfei Ma, Fangli Xu, Alex X. Liu, Chunming Wu, and Shouling Ji. 2021. Deep Graph Matching and Searching for Semantic Code Retrieval. *ACM Trans. Knowl. Discov. Data* 15, 5, Article 88 (may 2021), 21 pages. doi:10.1145/3447571
- [45] Chenxiao Liu, Shuai Lu, Weizhu Chen, Daxin Jiang, Alexey Svyatkovskiy, Shengyu Fu, Neel Sundaresan, and Nan Duan. 2023. Code Execution with Pre-trained Language Models. In *Findings of the Association for Computational Linguistics: ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 4984–4999. doi:10.18653/v1/2023.findings-acl.308
- [46] Chao Liu, Xin Xia, David Lo, Cuiyun Gao, Xiaohu Yang, and John Grundy. 2021. Opportunities and Challenges in Code Search Tools. *ACM Comput. Surv.* 54, 9, Article 196 (Oct. 2021), 40 pages. doi:10.1145/3480027
- [47] Chao Liu, Xin Xia, David Lo, Zhiwei Liu, Ahmed E Hassan, and Shanping Li. 2021. Codematcher: Searching code based on sequential semantics of important query words. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 1 (2021), 1–37.
- [48] Sirui Liu, Weirong Guo, Yaoxiang Yu, and Bo Cai. 2023. Enhancing Code Search with Token-Level Information Flow Graphs Generated from Aligned Structural and Textual Features. In *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2283–2288. doi:10.1109/SMC53992.2023.10394423
- [49] Shangqing Liu, Xiaofei Xie, Jingkai Siow, Lei Ma, Guozhu Meng, and Yang Liu. 2023. GraphSearchNet: Enhancing GNNs via Capturing Global Dependencies for Semantic Code Search. *IEEE Trans. Softw. Eng.* 49, 4 (apr 2023), 2839–2855. doi:10.1109/TSE.2022.3233901
- [50] Meili Lu, Xiaobing Sun, Shaowei Wang, David Lo, and Yucong Duan. 2015. Query expansion via wordnet for effective code search. In *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, 545–549.
- [51] Fei Lv, Hongyu Zhang, Jian-guang Lou, Shaowei Wang, Dongmei Zhang, and Jianjun Zhao. 2015. Codehow: Effective code search based on api understanding and extended boolean model (e). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 260–270.
- [52] Zheng Ma, Yuexiu Gao, Lei Lyu, and Chen Lyu. 2022. MMF3: Neural Code Summarization Based on Multi-Modal Fine-Grained Feature Fusion. In *Proceedings of the 16th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (Helsinki, Finland) (ESEM ’22)*. Association for Computing Machinery, New York, NY, USA, 171–182. doi:10.1145/3544902.3546251

- [53] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. 2016. Convolutional Neural Networks over Tree Structures for Programming Language Processing. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence, February 12-17, 2016, Phoenix, Arizona, USA*, Dale Schuurmans and Michael P. Wellman (Eds.). AAAI Press, 1287–1293. <http://www.aaai.org/ocs/index.php/AAAI/AAAI16/paper/view/11775>
- [54] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.
- [55] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2009. The Graph Neural Network Model. *IEEE Transactions on Neural Networks* 20, 1 (2009), 61–80. doi:10.1109/TNN.2008.2005605
- [56] Yucen Shi, Ying Yin, Zhengkui Wang, David Lo, Tao Zhang, Xin Xia, Yuhai Zhao, and Bowen Xu. 2022. How to better utilize code graphs in semantic code search?. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Singapore) (ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 722–733. doi:10.1145/3540250.3549087
- [57] Jianhang Shuai, Ling Xu, Chao Liu, Meng Yan, Xin Xia, and Yan Lei. 2020. Improving code search with co-attentive representation learning. In *Proceedings of the 28th International Conference on Program Comprehension*. 196–207.
- [58] Zeyu Sun, Qihao Zhu, Yingfei Xiong, Yican Sun, Lili Mou, and Lu Zhang. 2020. TreeGen: A Tree-Based Transformer Architecture for Code Generation. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, New York, NY, USA, February 7-12, 2020*. AAAI Press, 8984–8991. <https://aaai.org/ojs/index.php/AAAI/article/view/6430>
- [59] Martin Sundermeyer, Ralf Schlüter, and Hermann Ney. 2012. Lstm neural networks for language modeling.. In *Interspeech*, Vol. 2012. 194–197.
- [60] Kai Sheng Tai, Richard Socher, and Christopher D. Manning. 2015. Improved Semantic Representations From Tree-Structured Long Short-Term Memory Networks. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Chengqing Zong and Michael Strube (Eds.). Association for Computational Linguistics, Beijing, China, 1556–1566. doi:10.3115/v1/P15-1150
- [61] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (Long Beach, California, USA) (NIPS’17)*. Curran Associates Inc., Red Hook, NY, USA, 6000–6010.
- [62] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=rJXMpikCZ>
- [63] Yao Wan, Jingdong Shu, Yulei Sui, Guandong Xu, Zhou Zhao, Jian Wu, and Philip Yu. 2019. Multi-modal Attention Network Learning for Semantic Source Code Retrieval. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 13–25. doi:10.1109/ASE.2019.00012
- [64] Chong Wang, Xin Peng, Zhenchang Xing, Yue Zhang, Mingwei Liu, Rong Luo, and Xiujie Meng. 2023. XCoS: Explainable Code Search Based on Query Scoping and Knowledge Graph. *ACM Trans. Softw. Eng. Methodol.* 32, 6, Article 140 (Sept. 2023), 28 pages. doi:10.1145/3593800
- [65] Xin Wang, Yasheng Wang, Fei Mi, Pingyi Zhou, Yao Wan, Xiao Liu, Li Li, Hao Wu, Jin Liu, and Xin Jiang. 2021. Syncobert: Syntax-guided multi-modal contrastive pre-training for code representation. *arXiv preprint arXiv:2108.04556* (2021).
- [66] Ling Xu, Huanhuan Yang, Chao Liu, Jianhang Shuai, Meng Yan, Yan Lei, and Zhou Xu. 2021. Two-Stage Attention-Based Model for Code Search with Textual and Structural Features. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 342–353. doi:10.1109/SANER50967.2021.00039
- [67] Chen Zeng, Yue Yu, Shanshan Li, Xin Xia, Zhiming Wang, Mingyang Geng, Linxiao Bai, Wei Dong, and Xiangke Liao. 2023. deGraphCS: Embedding Variable-based Flow Graph for Neural Code Search. *ACM Trans. Softw. Eng. Methodol.* 32, 2, Article 34 (2023), 27 pages. doi:10.1145/3546066
- [68] Jingfeng Zhang, Haiwen Hong, Yin Zhang, Yao Wan, Ye Liu, and Yulei Sui. 2021. Disentangled Code Representation Learning for Multiple Programming Languages. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.). Association for Computational Linguistics, Online, 4454–4466. doi:10.18653/v1/2021.findings-acl.391
- [69] Wei Zhao and Yan Liu. 2022. Utilizing Edge Attention in Graph-Based Code Search.. In *SEKE*. 60–66.
- [70] Zixiang Zhao, Haowen Bai, Jiangshe Zhang, Yulun Zhang, Shuang Xu, Zudi Lin, Radu Timofte, and Luc Van Gool. 2023. Cddfuse: Correlation-driven dual-branch feature decomposition for multi-modality image fusion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 5906–5916.
- [71] Daniel Zügner, Tobias Kirschstein, Michele Catasta, Jure Leskovec, and Stephan Günnemann. 2021. Language-Agnostic Representation Learning of Source Code from Structure and Context. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. <https://openreview.net/forum?id=Xh5eMZVONGF>

Received 7 December 2024; revised 29 June 2025; accepted 20 August 2025